



Rapport de Stage

« Mastère en Langue et Informatique »

Présenté par

Yaqi DENG

Matching Ontology basée sur WordNet et MeSH & Implémentation de Browser et de REST API Lexicon

Rapport Dirigé par :

Mme Victoria EYHARABIDE

Encadrement Académie

M. Hugues de MAZANCOURT

Encadrement Société

Soutenue le 25/09/2019
Au sein du Maison de la Recherche

L'année Mastère 2018 - 2019

Remerciements

Je tiens à exprimer toute ma reconnaissance à ma directrice de rapport de stage, Madame Victoria EYHARABIDE. Je la remercie de m'avoir encadré, orienté, aidé et conseillé.

Je voudrais remercier sincèrement Monsieur Hugues de MAZANCOURT, qui, m'a prodigué de nombreux conseils et m'a formulé des propositions très utiles.

J'adresse mes sincères remerciements à tous les professeurs, intervenants et toutes les personnes qui par leurs paroles, leurs écrits, leurs conseils et leurs critiques ont guidé mes réflexions et ont accepté de me rencontrer et de répondre à mes questions durant la rédaction ce rapport de stage.

Résumé

Au cours de ce stage, j'ai énormément appris dans les missions enrichissantes que l'entreprise m'a demandé. Dans une condition que l'entreprise travaille pour les assurances, l'entreprise aura besoin de "common knowledge" en médecine pour le grand public, où j'ai eu une opportunité à faire l'enrichissement pour nos bases de données avec différents manières, soit à la manière de informatique, par exemple réaliser un browser et un API afin une maintenance de dictionnaire, soit à la manière de recherche, par exemple établir une version de Matching Ontology afin d'ajouter des "common knowledge" en médecine pour le grand public. Pour réaliser un browser et un API, j'avais utiliser Hibernate et modèle Repository, une manipulation de ces deux outils en JAVA approfondit mon compétence et la manière de pensée en information, deux outils ne sont pas difficile mais bien pour une élève sortante. Après deux mois d'implémentation, j'ai pu fini cette mission avec deux méthode GET et POST dans browser tel que les fonctions comme "find word by lemma", "add word", etc. Ainsi que quatre méthode GET, POST, PUT, DELETE dans API tel que les fonctions "Add, Find, Update and Delete". Ensuite, Le Matching Ontology, on a réalisé une version de Matching pendant deux mois. Pour les "common knowledge" en médecine soit compréhensible pour le grand public, on a séparé cette mission en trois grands phrases : *Recherche L'état de l'art*, *Réalise le Matching Ontology*, *réflexion d'Intégration Ontology*. Dans premier phrase, j'ai pu trouver quatre approches ressemble à notre situation, tel que une approche de X-similarité, une approche de Ontology Reuse, une approche de S-Match et une approche de la variant d'approche de S-Match. Dans deuxième phrase, j'ai pu réaliser une version de Matching qui basé sur WordNet et MeSH, avec une approche hybride et la réalisé une étape après une étape de problématique rencontré, à travers trois algorithmes basé sur les techniques basé sur la livre « Ontology Matching ». Tel que Égalité de chaîne, la Similarité Sémantique ainsi que comparaison le premier ancêtre concept commun entre deux ontologie, on considère 2 468 concepts MeSH sont intégrable dans WN et ceux qui attendre une série d'algorithme pour déterminer quel parti de ces concepts en médecine peuvent être intégrées. Finalement, dans la troisième phrase, la réflexion d'Intégration, cette phrase est concerné la création d'ontologie. Dans une limite que le temps et les ressources humains ne sont pas assez, tous ces tâches sont réaliser avec une manière parcimonie. Le reste du temps m'a permit de construire cet rapport de stage.

Table des matières

Remerciements	1
Résumé	1
1 Introduction	1
1.1 Problématique soulevée dans Matching Ontology	5
1.2 Organisation du rapport	6
2 Browser et API Lexicon	8
2.1 Les bases données utilisées	8
2.2 Les Relations Hibernation Utilisées	9
2.3 Modèle de Repository	10
2.4 L'implémentation de Browser	11
2.5 L'implémentation de REST API	15
3 État de l'art : La Mesure Similarité Sémantique en ontologie croisée	17
3.1 Matching Ontology	17
3.1.1 Type d'hétérogène	18
3.1.1.1 Conceptuelle hétérogène	18
3.1.1.2 Sémiotique hétérogénéité	19
3.1.2 Technique Élémentaire utilisée pour Matching Ontologie	19
3.1.2.1 Techniques basées sur les chaînes d'identifiant	19
3.1.2.2 Techniques basées sur le langage	19
3.1.2.3 Techniques basées sur les graphes	20
3.2 La Mesure Sémantique	20
3.2.1 La Sémantique Similarité	21
3.2.2 Quatre branches de Similarité Sémantiques	21
3.3 État de l'art	22
3.3.1 X-similarité [5]	22
3.3.2 An Approach to Ontology Integration for Ontology Reuse (Enrico G. Caldarola, 2016)[4]	23
3.3.2.1 Méthodologie de Matching dans cet article	24
3.3.3 S-Match : an algorithm and an implementation of semantic matching(Giunchiglia, 2004)[6]	26
3.3.4 L'Approche variante de S-Match : Semantic Matching Using the UMLS(S-Match)(Shamdasani, 2009) [7]	28
3.3.4.1 La modification d'algorithme de S-Match	29

4	Approche utilisée	31
4.1	Les bases données utilisées	31
4.2	Le Matching d'Ontologie	33
4.2.1	Premier algorithme : Égalité de chaîne	36
4.2.1.1	Technique basée sur des chaînes et basées sur la langue	36
4.2.1.2	Égalité des chaînes	37
4.2.2	Deuxième algorithme : Similarité sémantique	38
4.2.2.1	Prétraitement des données	39
4.2.2.2	Entraînement des modèles	40
4.2.3	Troisième algorithme : comparaison des concepts voisins	43
4.3	Résultat de Matching	45
4.4	Évaluation de Matching	46
4.5	Réflexion de l' intégration	47
4.6	Évaluation via Recherche d'information	48
4.7	Conclusion	49
5	Typologie	50
5.1	Typologie 1 – Concept MeSH comme instance dans WordNet	50
5.2	Typologie 2 – Concept MeSH et WordNet ont différent concept hyperonyme	50
5.3	Typologie 3 - Concept a hyponyme	51
5.4	Typologie 4 - Comparaison entre premier ancêtre concept WordNet et tous les ancêtres concepts MeSH	51
5.5	Typologie 5 – La troisième forme d'identifiant de concept MeSH	51
5.6	Typologie 6 - La quatrième forme d'identifiant de concept MeSH	52
5.7	Typologie 7 - Out of Vocabulary(OOV) durant l'entraînement de modèle dans le deuxième algorithme	52
6	Conclusion	53
	Annexes	55
	Annexe 1	55
1	JPA utilisé dans browser	55
2	Status utilisé dans API	55
	Annexe 2	57
1	Terminologie d'ontologie	57
2	Technique élémentaire Matching Ontologie	59
2.1	Techniques basées sur les chaînes	59
2.2	Techniques basées sur le langage	59
2.3	Techniques basées sur les contraintes	60
2.4	Techniques informelles basées sur les ressources	60
2.5	Techniques basées sur les ressources formelles	60
3	Techniques au niveau de la structure	60
3.1	Techniques basées sur les graphes	60
3.2	Techniques basées sur la taxonomie	61
3.3	Techniques basées sur des modèles	61

3.4	Techniques basées sur des instances	61
4	L'état de l'art d'une même ontologie	61
4.1	Approche basée sur Structure des Graphes	62
4.1.1	Les Méthodes Variantes	62
4.1.2	Shortest Path	63
4.1.3	Weighted Links	64
4.1.4	Hirst and St-Onge Measure	64
4.1.5	Leacock et Chodorow	64
4.2	Approche basé sur les caractéristiques du concept	65
4.2.1	Tversky (1977) [22]	66
4.3	Approche basé sur la théorie de l'information	66
4.3.1	Les méthodes variantes	66
4.3.2	Lin	67
4.4	Approches hybrides	67
4.4.1	Knappe	67
4.4.2	Rodriguez et al.	68

Chapitre 1

Introduction

Yseop¹ est un éditeur français de logiciels d'intelligence artificielle, leader technologique dans *Natural-language generation*(NLG). L'entreprise est spécialisée dans la conception de produits permettant la compréhension du langage naturel dans le secteur de l'intelligence artificielle. La solution de NLG proposée par Yseop est capable de raisonner et de rédiger en plusieurs langues. Elle peut ainsi transformer automatiquement des données en textes compréhensibles par tous.

L'entreprise contient six départements qui sont YCS (customer success), RD, Marketing, Ventes, Administratif, Lab, et environ 80 salariés. J'ai intégré les départements de *Lab* et de *Research and Development*(R & D). Au cours du stage que j'ai réalisé au sein de l'entreprise Yseop de mars 2019 à septembre 2019 (pour une durée de cinq mois), j'ai interagi principalement avec l'équipe *Natural-language understanding*(NLU), ceux qui sont mon tuteur et avec un autre collègue.

L'équipe NLU a pour mission de développer les composants de NLU destinés aux chatbots dans les domaines de la finance et de la médecine, par exemple, un des produits de la société *Smart Personal Advisor*(SPA) : Yseop utilise les données du CRM pour fournir aux équipes commerciales des renseignements et indications leur permettant de proposer le bon produit au bon client. Étant donné que les projets d'entreprise concernent également les assurances et les pharmacies, l'entreprise a besoin de "common knowledge" en médecine pour le grand public. Même si les besoins dans ces deux domaines ne sont pas exactement les mêmes, on pourra les faire évidemment converger.

Pour répondre à ces besoins, mes missions sont construites autour de deux axes : le premier axe est d'enrichir notre dictionnaire en ajoutant les lexiques dans le domaine biomédical. Cette tâche consiste principalement à mener des campagnes systématiques d'amélioration à partir de ressources externes (banques de données terminologiques, dictionnaires ouverts, etc). Nous avons choisi d'enrichir notre dictionnaire basé sur le domaine d'ontologie.

Une ontologie est « une spécification explicite d'une conceptualisation » (Gruber, 1993), une collection de faits concernant un domaine d'entités définies. Les on-

1. <https://yseop.com/fr/>

tologies varient selon différentes dimensions, notamment leur taille, leur complexité, leur domaine et leur spécificité. Certaines de leurs règles sont des formules logiques complexes, alors que d'autres sont des ensembles davantage superficiels de classes. Il est entendu que les concepts ou les termes caractérisant différentes sources de communication (par exemple, les humains ou les ordinateurs) sont représentés par des ontologies. Il est courant que les sources communicantes utilisent la même ontologie. Cependant, il est également possible qu'une ontologie différente soit utilisée pour représenter les concepts de chaque source de connaissance. À cette fin, une ontologie est organisée par concept et en identifie toutes les relations possibles entre eux. Une fois qu'un domaine de la connaissance ou certains aspects sont officiellement représentés à l'aide d'un langage commun et partagé, ils deviennent compréhensibles non seulement par l'homme, mais également par des agents informatisés. Pour ces raisons, les ontologies sont de plus en plus considérées comme un facteur clé pour permettre l'interopérabilité de systèmes hétérogènes et améliorer l'efficacité du processus de représentation et de récupération des connaissances.

Notre dictionnaire est basé sur *Lexique des Formes Fléchies du Français* (Leff) et *WordNet Libre du Français* (WOLF)¹, Ainsi, nous avons décidé de faire un *Matching Ontology*² pour enrichir notre dictionnaire dans le domaine du biomédical. La valeur du matching ontology réside dans le fait qu'il existe de nombreuses terminologies, taxonomies et ontologies largement indépendantes qui couvrent les différents sous-domaines de la médecine. Afin que l'interopérabilité entre ces modèles sémantiques soit réalisée, ils doivent être adaptés. Ainsi, la ressource de notre approche basée sur *WordNet* et *Medical Subject Headings* (MeSH), la raison pour laquelle nous avons choisi MeSH, car pour des ontologies bien structurées telles que MeSH ou Unified Medical Language System (UMLS), les communautés de domaine correspondantes peuvent atteindre un consensus sur la connaissance dans les ontologies. Pour cette raison, MeSH peuvent être utilisées en tant que connaissances de domaine pour des systèmes basés sur la connaissance ou des agents conversationnels comme les produits de notre entreprise. Ontologie MeSH n'autorise pas un usage commercial. Toutefois les données sont bien structurées, ce qui permet de faire un matching sur ces deux ontologies. Ces expériences pourront être utiles à l'avenir sur l'autre ontologie, l'algorithme conçoit et réalise sur les techniques de matching ontology. Après, une réflexion d'intégration d'ontologie à la fin de ce Matching. Ainsi, la première partie de mission fut d'enrichir la partie WordNet à partir du MeSH.

Pour réaliser ce matching ontology, j'ai, en premier lieu, établi un état de l'art de l'intégration ou reuse d'ontologie en utilisant la *Sémantique Similarité*³. L'état de l'art sera discuté selon les notions nécessaires pour le matching ontology ainsi que les notions

1. Concerne la présentation de ces deux lexicons réalisée par (Sagot et al.), qui sera décrite dans le chapitre II - Browser et Api Lexicon.

2. Correspond au processus de recherche de relations ou de correspondances entre des entités d'ontologies différentes. Il existe plusieurs termes dans ce domaine : mapping ontology, matching ontology, alignement ontology etc. Les notions de tous ces termes sont présentées dans l'annexe 2 - Terminologie d'ontologie. Nous avons choisi ce terme car il ressemble beaucoup à notre situation.

3. L'approche principalement utilisée dans la communauté. La similarité sémantique concerne le calcul de la similarité entre des concepts qui ne sont pas forcément semblables lexiquement.

de la sémantique similarité¹. La raison pour laquelle nous avons choisi ces approches est que ces quatre approches sont similaires à la notre. Après une étude de l'état de l'art dans la communauté de matching ontology, nous avons décidé de réaliser ce matching basé sur le type d'entité de concept dans lequel ils co-existent dans WordNet et MeSH. En effet, il existe un certain nombre d'approches de matching ontology basées sur l'entité de concept. Le type d'entité basé sur le concept nous permet de réaliser un matching indépendant de la langue comme la solution de NLG proposée par Yseop. Même s'il existe des problématiques comme des concepts qui n'existent pas dans certains pays mais qui existent dans d'autres, le type d'entité basé sur entité concept est quand même mieux que basé sur entité langue. En utilisant la baseline² de ce matching d'ontologie, j'ai pu trouver les concepts intégrables ainsi que mener une réflexion sur quelle partie des concepts biomédicaux, qui est la partie générale, peut s'intégrer au concept général en utilisant WordNet³ comme ontologie de référence.

Pour clarifier son sens, et parce que cette mission discute beaucoup de la notion de "concept", nous allons présenter ici brièvement la notion de concept. Le terme « concept » a été utilisé pour désigner de nombreux phénomènes. Selon l'étude pionnière de Boden (1990) sur la créativité, un concept est une idée abstraite des arts, des sciences et de la vie quotidienne. MeSH est une hiérarchie taxonomique de termes médicaux et biologiques proposée par la US National Library of Medicine (NLM). Le vocabulaire contrôlé contient plusieurs sortes de termes, tels que descripteur, qualificatifs, types de publication, données géographiques et termes de saisie etc. Dans MeSH, chaque descripteur comprend un ou plusieurs concepts, et chaque concept consiste en un ou plusieurs termes synonymes. Par exemple :

Cardiomegaly [Descriptor]	
Cardiomegaly	[Concept, Preferred]
Cardiomegaly	[Term, Preferred]
Enlarged Heart	[Term]
Heart Enlargement	[Term]
Cardiac Hypertrophy	[Concept, Narrower]
Cardiac Hypertrophy	[Term, Preferred]
Heart Hypertrophy	[Term]

FIGURE 1.1 – Structure un descripteur dans MeSH

Ce descripteur comprend deux concepts et cinq termes. Chaque concept a un terme

-
1. Ce qui est principalement utilisé pour notre deuxième algorithme ainsi que pour la plupart des autres approches qui utilisent cette méthode.
 2. Le code Python réalisé par moi-même en suivant notre approche.
 3. WordNet a été créé dans le but d'améliorer les modèles psycholinguistiques d'organisation des concepts. La base de données lexicale électronique est devenue de plus en plus populaire parmi les experts en Traitement Automatique du langage naturel (TAL) qui traitent le sens des mots et leurs relations, ainsi que parmi les experts en ontologie. WordNet peut être considéré comme une ontologie qui pondère les mots, les sens et une série de relations entre eux, mais qui ressemble davantage à une ontologie qu'à, par exemple, un dictionnaire lisible par machine ou un thésaurus. Nous pourrions considérer les synsets WordNet comme un concept, ainsi, 117 000 synsets existent dans WordNet.

préféré, qui est également appelé l'identifiant du concept, et chaque descripteur a un concept préféré. Le nom du descripteur — le terme le plus utilisé pour désigner le descripteur — est le terme préféré du concept préféré. Dans chaque concept, les termes sont synonymes. En revanche, tout comme pour le descripteur, les termes d'un concept ne sont pas forcément les synonymes des termes d'un autre concept. Ainsi, un concept dans un descripteur peut être plus étroit que le concept préféré. Nous pouvons également noter que les termes d'un concept sont issus de cette relation et sont donc plus proches, par exemple, que les termes de l'autre concept. Cependant, tous les termes d'un descripteur sont équivalents aux fins de l'indexation et de la recherche dans MEDLINE. Ils constituent donc toujours des termes d'entrée pour le descripteur. Pour la partie d'extraction de concepts à partir de MeSH, on a extrait simplement le concept préféré cela veut dire le descripteur ou bien l'identifiant de concept afin que cela ne rende pas le sujet encore plus difficile.

WordNet se présente un concept comme un synset, un ensemble de synonymes cognitifs. Dans WordNet, la relation principale entre les mots est la synonymie, notamment entre les mots fermés tels que voiture et automobile. Les synonymes — les mots qui dénotent le même concept et sont interchangeables dans de nombreux contextes — sont regroupés dans des ensembles non ordonnés (synsets). Chacun des 117 000 synsets de WordNet est lié à d'autres au moyen d'un petit nombre de « relations conceptuelles ». En outre, un synset contient une brève définition « gloss » et, dans la plupart des cas, une ou plusieurs phrases illustrant son utilisation. Les formes de mots avec plusieurs significations distinctes sont représentées dans autant de synsets distincts. Ainsi, chaque paire forme-signification dans WordNet est unique. Pour la partie d'extrait de concept WordNet, on a utilisé l'interface de WordNet et l'identifiant de concept MeSH ce qu'on a extrait. C'est à dire, on utilise l'identifiant MeSH pour trouver les concepts WordNet similaire, on présentera cette partie dans la chapitre IV - Approche utilisé.

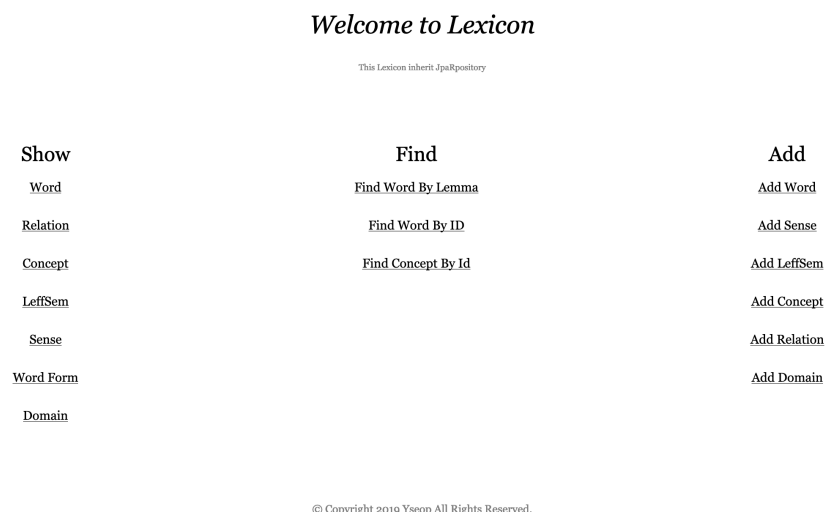


FIGURE 1.2 – Un aperçu de browser réalisé

Ma deuxième mission est d'établir un browser et API afin d'effectuer la maintenance

du dictionnaire et l'ajout de nouvelles composantes à celui-ci. En effet, nous voulions avoir une couche d'abstraction qui permette de manipuler des objets linguistiques (mots, sens, concepts, ...) indépendamment de la manière dont ils sont représentés dans la base de données. Ce browser est réalisé avec JAVA EE, construit avec Apache Maven¹, encadré avec Spring boot. Pour la partie de liaison entre base de données et browser, nous utilisons Hibernate en tant que Mapping objet-relationnel (ORM), Freemarker² et HTML comme moteur de fond et CSS pour l'affichage. Enfin, notre base de données est stockée dans MySQL.

Toutes ces missions ont été pour moi l'occasion de percevoir le croisement de sciences comme la première mission : le matching d'ontologie qui intègre non seulement le domaine de l'informatique à la pensée philosophique. Nous pouvons, par exemple, nous interroger sur la notion de concept, et dans quel cas faut-il intégrer deux concepts qui sont interprétés par deux experts, etc. Pour la deuxième mission, il en a été de même dans le cadre d'une autre mission : établir un browser nécessite une compétence en informatique, mais aussi de réfléchir au niveau linguistique sur la relation entre les mots, les sens, les concepts, la forme des mots, etc.

Ces missions ont été mises en place dans l'objectif de réaliser une version complète qui comprendrait le *Matching*, une réflexion d'*Intégration Ontologie* et une proposition sur l'*Évaluation* de la qualité de l'intégration ontologie. Si tous les cas n'ont pas été traités dans chacune de ces trois grandes phrases, nous en présentons la possibilité dans chaque partie ou dans l'annexe. Nous reviendrons sur ces notions dans chaque partie de matching d'ontologie. Par conséquent, nous avons décidé d'implémenter notre baseline en utilisant Python aux méthodes et tests afin de diminuer le temps de réalisation, ainsi que des bibliothèques abandonnées pour effectuer des recherches plus rapidement. Tout au long de mon stage, j'ai pu apprendre dans d'excellentes conditions grâce à mon maître de stage Hugues De Mazancourt, VP (Vice Président Innovation) de l'entreprise. Ce dernier a su m'apporter un soutien de qualité et me guider patiemment dans les missions qui sont parfois difficiles.

1.1 Problématique soulevée dans Matching Ontology

Aujourd'hui, le matching d'ontologie demeure un sujet ouvert, en particulier auprès des communautés d'ontologie qui font, en majeure partie, une évaluation Gold standard³. En ce qui concerne les ressources humaines, nous n'étions que deux personnes pour réaliser cette mission. Nous n'avions que deux mois environ pour effectuer une version de matching, ce qui augmente encore la difficulté de cette mission. Heureusement, ces deux ontologies sont bien structurées et ont un consensus d'appréciation dans les communautés.

1. Couramment appelé Maven, Apache Maven est un outil de gestion et d'automatisation de production des projets logiciels Java en général et Java EE en particulier.

2. Freemarker est un moteur de templates ou gabarits qui permet de générer automatiquement des fichiers textes à partir d'un modèle de données de type tableau associatif. Il s'agit d'une bibliothèque logicielle écrite en Java, souvent utilisée pour générer des pages Web ou du code source.

3. Annotation manuelle

Afin d'obtenir au moins une version assez complète et correcte, nous proposons ici la première problématique : comment matcher deux ontologies avec parcimonie tout en obtenant un résultat satisfaisant ? Pour y parvenir, nous avons appliqué les méthodes les plus réalisables afin d'archiver cette version de Matching et de nous laisser un peu de temps pour les affiner et une réflexion d'intégration. Finalement, nous avons décidé de nous baser sur une façon de faire qui nous permettrait d'épargner nos efforts et de gagner du temps, tout en obtenant un résultat satisfaisant, au fur et à mesure du matching. Il serait intéressant de résoudre les problématiques soulevées ainsi que les typologies et difficultés rencontrées. Finalement, nous avons trois algorithmes : un basé sur les chaînes, un basé sur la similarité sémantique et l'autre basé sur les graphes. Notre méthode est basée principalement sur l'approche de la Mesure Sémantique(MS) et nous permet d'aligner deux concepts non seulement au niveau de morphologie mais aussi au niveau sémantique et aussi au niveau structure de graphe.

Une deuxième problématique est alors apparue concernant l'évaluation de la qualité du matching : quelle méthode choisir pour que l'évaluation soit fiable et appréciée ? L'évaluation Gold standard de ce genre de matching nécessite au moins un expert en biomédical. Or, l'entreprise n'en dispose d'aucun. Nous avons donc pensé à réaliser cette évaluation via un autre algorithme. Pour la solution, nous avons discuté de deux pistes avec mon tuteur et mon encadrement. L'une des pistes est une évaluation semi-automatique basée sur l'apprentissage supervisé. L'autre piste est une évaluation automatique basée sur Recherche Information en utilisant les données médicales de « HealthTap » par le biais de 1 605 232 questions et réponses. Les détails seront présentés à la fin du Chapitre Approche Utilisée. L'évaluation n'a pas été réalisée en raison de temps, néanmoins, nous notons quand même à la fin de la chapitre IV - Approche utilisée.

La dernière problématique mettre en question à l'étape d'intégration d'ontologie : Comment pourrions-nous intégrer la partie générale du concept MeSH dans WordNet ? Nous avons donc réfléchi à cette intégration après avoir fini une version de matching, puis nous avons pensé à proposer une évaluation de deux manières automatiques : basée sur la Recherche d'information(RI) et sur la Matching Learning(ML). Étant donné la quantité de travail, nous n'avons pas mis en place la solution d'intégration et d'évaluation dans notre baseline, mais nous notons notre pensée de ces deux parties à la fin de "Chapitre IV - Approche Utilisée" afin du travail à l'avenir. Pour la solution, nous avons présenté le but final de cette mission qui est d'intégrer la partie générale du concept médical dans notre base de données dans le domaine général. Ainsi, dans la première partie, nous traiterons de la partie générale des concepts médicaux. Cette solution concerne la création d'ontologie et la réutilisation d'ontologie, car nous devrions définir la généralité d'un concept du domaine « général ». Nous préciserons les détails également à la fin du "Chapitre IV - Approche Utilisée".

1.2 Organisation du rapport

Afin de rendre compte de manière analytique des cinq mois passés au sein de la société Yseop, il me semble logique de présenter au préalable l'environnement du stage,

à savoir le secteur du NLG, ainsi que les missions et les problématiques, etc. (Chapitre I - Introduction)

Dans un premier temps, nous présenterons le Browser et l'API que j'ai réalisés, y compris les dessins, les idées, les librairies utilisées, etc. Lors de la présentation des Browser et API, nous expliquerons comment établir les liens entre les tableaux de mot, de sens, de concept, etc en utilisant Hibernate et Modèle Repository. En ce qui concerne la partie API, j'ai réalisé une API REST (Representational state transfer)¹. Mon but est de dessiner une branche facile à comprendre et un browser simple à utiliser. (Chapitre II - Browser Lexicon et API Lexicon)

Par la suite, nous étudierons la mission principale, le matching d'ontologie, et proposerons un état de l'art sur la mesure similarité sémantique en ontologie croisée avant de nous intéresser à l'approche utilisée pour le matching et l'implémentation. Dans ce chapitre, nous présenterons également les notions de matching d'ontologie et les notions de la mesure de la similarité sémantique, à savoir les concepts extraits à partir de WordNet et MeSH ainsi que leurs structures. Ces dernières sont alors nécessaires afin de construire une base de connaissances pour ceux qui apparaissent pour la première fois dans cet aspect ou pour réviser ce qui est déjà compris dans cet aspect. La première étape pour le matching d'ontologie permet donc de comprendre la base. (Chapitre III - État de l'art)

Après avoir eu un aperçu de cet état de l'art, nous avons décidé de définir les concepts qui co-existent à partir de trois algorithmes et de les implémenter l'un après l'autre. Ainsi, les concepts extraits par ces trois algorithmes ont trois genres d'attribut permettant de les utiliser pour la partie d'intégration tels que les features de concepts (il existe sept colonnes d'information), les sens de concepts paires (extraits à travers la sémantique similarité), les pères concepts de concepts paires (extraits à travers une comparaison d'ancêtre concept de concept paire). En début de chapitre, nous présenterons la méthode de matching avant d'exposer les résultats apparus et après avoir réalisé la première étape de chacun des trois algorithmes. (Chapitre IV - Approche Utilisée)

Au travers des concepts intégrables que nous avons trouvés, nous rencontrons les typologies concrètes concernant le matching d'ontologie et l'évaluation de matching. Par exemple, les concepts de MeSH et de WordNet ont différents concepts voisins. Nous pouvons donc nous demander si ces derniers peuvent être intégrés. (Chapitre V - Typologie)

Enfin, en conclusion générale, nous verrons tout ce que ces missions m'ont apporté durant ce stage. (Chapitre VI - Conclusion)

1. REST est un style d'architecture logicielle définissant un ensemble de contraintes à utiliser pour créer des services web. Les services web conformes au style d'architecture REST, aussi appelés services web RESTful, établissent une interopérabilité entre les ordinateurs sur Internet.

Chapitre 2

Browser et API Lexicon

Dans ce chapitre, nous présenterons les bases de données utilisées dans ce Browser¹ et API², la ressource de notre dictionnaire vient principalement de WOLF et Leff³ [17]. Par la suite, nous nous intéresserons à la réalisation et aux idées principales. Afin de réorganiser les objets linguistiques et de concevoir un browser clair, nous établirons les relations et les caractéristiques de ces tableaux, etc. Enfin, nous proposerons un aperçu de browser et terminerons sur une petite discussion.

2.1 Les bases données utilisées

Les bases de données utilisées dans ce browser, dont la ressource de notre dictionnaire, sont principalement tirées de Leff et WOLF. Leff est développé avec une architecture pour les lexiques informatiques et leur acquisition (Alexina³), qui comprend deux niveaux : le lexique intentionnel et le lexique d'extension. Le premier comprend une forme canonique et une forme inflexion pour chaque entrée d'un terme donné, à savoir des informations syntaxiques profondes. La deuxième sorte de lexique est construite automatiquement lors de la compilation du premier. WOLF est une ressource lexicale sémantique gratuite pour le français. Il a été créé à partir de Princeton WordNet et diverses ressources multilingues (Sagot et Fišer 2008a, Sagot et Fišer 2008b, Fišer et Sagot 2008).

À partir de ces deux principes de ressources lexiques, nous avons construit nos six tableaux : word, leff_sem, wordform, sense, rel, concept. « Word » correspond à l'entrée qui permet de chercher dans les autres tableaux les caractéristiques d'un terme, la langue de ce terme, la POSTag⁴ et le modal, etc.

1. Type commun de client Web. Tim Berners-Lee a développé le premier, capable de visualiser et de modifier des documents HTML.

2. Utilisé par les clients pour interagir avec un service Web.

3. Alexina est un cadre d'acquisition et de modélisation lexicale couvrant à la fois les niveaux morphologique et syntaxique.

4. Étiquetage morpho-syntaxique

id	cpos	pos	lemma	lang	infl	tag	modal	neg	disabled	origin
55	ADV	ADV	à donf	fr	inv	NULL	0	0	0	lefff-3.4
56	ADV	ADV	à droite	fr	inv	NULL	0	0	0	lefff-3.4
57	ADV	ADV	à fond	fr	inv	NULL	0	0	0	lefff-3.4
58	ADV	ADV	à force	fr	inv	NULL	0	0	0	lefff-3.4
59	ADP	P	à force de	fr	prep-e	NULL	0	0	0	lefff-3.4
60	ADV	ADV	a fortiori	fr	inv	NULL	0	0	0	lefff-3.4
61	ADV	ADV	à gauche	fr	inv	NULL	0	0	0	lefff-3.4
62	ADV	ADV	à gogo	fr	inv	NULL	0	0	0	lefff-3.4
63	ADV	ADV	à gorge déployée	fr	inv	NULL	0	0	0	lefff-3.4
64	ADP	P	à hauteur de	fr	prep-e	NULL	0	0	0	lefff-3.4
65	ADV	ADV	à hue et à dia	fr	inv	NULL	0	0	0	lefff-3.4
66	ADJ	ADJ	à huis clos	fr	adj-1	NULL	0	0	0	lefff-3.4
67	ADV	ADV	à jamais	fr	inv	NULL	0	0	0	lefff-3.4
68	ADJ	ADJ	à jeun	fr	adj-1	NULL	0	0	0	lefff-3.4

FIGURE 2.1 – Tableau "word" de source Lefff-3.4

« leff_sem » est un flag hérité de l'import du Lefff, qui n'est plus utilisé actuellement. « wordform » désigne l'inflexion de terme, « sense » se rapporte au sens du terme, et forme le lien entre le concept et le terme, tandis que « rel » décrit la relation entre les concepts. Enfin, «concept» est un concept.

id	name	def	usages	origin	semclass	score	disabled
1	eng-30-00001740-a	(usually followed by `t...	able to swim // s...	wolf-1.4b	capability	3	0
2	eng-30-00001740-b	without musical acco...	they performed a...	wolf-1.4b	AcapellaMusic	4	0
3	eng-30-00001740-n	concept formulant la c...		wolf-1.4b	Entity	2	0
4	eng-30-00001740-v	draw air into, and exp...	I can breathe bet...	wolf-1.4b	Breathing	3	0
5	eng-30-00001837-b	in the Christian era; us...	in AD 200	wolf-1.4b	NULL	4	0
6	eng-30-00001930-n	an entity that has phys...		wolf-1.4b	Physical	4	0
7	eng-30-00001981-b	of the period coincidin...	in 200 CE	wolf-1.4b	NULL	4	0
8	eng-30-00002098-a	(usually followed by `t...	unable to get to t...	wolf-1.4b	capability	3	0
9	eng-30-00002137-n	a general concept for...		wolf-1.4b	Abstract	2	0
10	eng-30-00002142-b	before the Christian er...	in 200 BC	wolf-1.4b	NULL	4	0
11	eng-30-00002296-b	of the period before th...	in 200 BCE	wolf-1.4b	NULL	4	0
12	eng-30-00002312-a	facing away from the...	the abaxial surfa...	wolf-1.4b	PositionalAttribute	4	0
13	eng-30-00002325-v	undergo the biomedic...		wolf-1.4b	Breathing	4	0
14	eng-30-00002436-b	on the back of a horse	he rode horseba...	wolf-1.4b	On	4	0
15	eng-30-00002452-n	a separate and self-co...		wolf-1.4b	CorpuscularObject	3	0

FIGURE 2.2 – Tableau "concept" de source WOLF-1.4

2.2 Les Relations Hibernation Utilisées

Pour manipuler nos données dans MySQL, nous utilisons un outil d'*object Relational Mapping* (ORM¹) qui permet de manipuler facilement les données et d'assurer leur persistance. Il en existe plusieurs. Cependant, dans le cadre de cette mission, j'ai utilisé Hibernate que nous présenterons ci-après.

Hibernate est un Framework open source de ORM ou de persistance² objet de données qui permet de réaliser le mapping entre des objets Java et des bases de données

1. Le mappage objet/relationnel fournit une méthodologie et un mécanisme permettant aux systèmes orientés objet de conserver leurs données à long terme en toute sécurité dans une base de données, avec un contrôle transactionnel sur celles-ci, tout en les exprimant lorsque cela est nécessaire dans les objets de programme.

2. Un objet (instance d'une classe) est persistant.

relationnelles. Avec Hibernate, nous pouvons réaliser la persistance des données, c'est-à-dire transcrire des classes Java dans des tables de bases de données, et des types de données Java dans les types de données MySQL, et vice versa. Il existe deux manières pour Hibernate de persister des données, HTML et Annotation similaire JPA. Dans un but de clarté, nous avons utilisé Annotation pour persister les données ainsi que l'implémentation de relation entre les objets linguistiques. Nous présenterons ensuite les notions de relation dans Hibernate, puis détaillerons l'utilisation de ces relations entre les tableaux.

One-to-one[2] : Correspondance entre des données de tables de base de données dans laquelle un enregistrement de la première table est associé à un enregistrement d'une autre table. One-to-many ou Many-to-one[2] : Correspondance entre des données de tables de base de données dans laquelle un enregistrement de la première table est associé à plusieurs enregistrements d'une autre table. Et vice versa. Dans notre base de données, ces deux relations sont principalement utilisées, entre le tableau de Mot et la tableau de Forme de mot qui est une relation One-to-one unidirection, mais Many-to-one unidirection entre mots et forme de mot. Chaque wordform n'a qu'un seul ID de mot, et chaque mot a plusieurs formes de mot. Entre le tableau de Mot et le tableau de Sens qui est une relation One-to-one unidirection entre mot et sens, et Many-to-one unidirection entre sens et mot. Chaque mot n'a qu'un seul ID de sens, et chaque sens a plusieurs mots. Également pour le tableau de Sens et le tableau de Concept, chaque sens n'a qu'un seul ID de concept, et chaque concept a plusieurs sens.

Many-to-many[2] : Correspondance entre des données de tables de base de données dans lesquelles plusieurs enregistrements de la première table sont associés à plusieurs enregistrements d'une autre table et plusieurs enregistrements de cette table sont associés à plusieurs enregistrements de la première table. Une relation plusieurs à plusieurs se produit lorsque plusieurs enregistrements d'une table sont associés à plusieurs enregistrements d'une autre table. Les systèmes de bases de données relationnelles ne nous permettent généralement pas d'implémenter une relation directe plusieurs-à-plusieurs entre deux tables. Pour éviter ce problème, nous pouvons diviser la relation plusieurs à plusieurs en deux relations un à plusieurs à l'aide d'une troisième table, appelée table de jointure. Chaque enregistrement d'une table de jointure comprend un champ de correspondance contenant la valeur des clés primaires des deux tables qu'il joint. (Dans la table de jointure, ces champs de correspondance sont des clés étrangères.) Ces champs de clés étrangères sont renseignés avec des données lorsque les enregistrements de la table de jointure sont créés à partir de l'une ou l'autre des tables qu'il joint. Cette relation se sert entre le tableau de Concept et le tableau de Relation, c'est-à-dire concept source et concept cible sont liés via une ou plusieurs relations. Ainsi, Many-to-many bidirection entre deux concepts. Chaque concept source a plusieurs relations avec le concept cible et chaque concept cible a plusieurs relations avec le concept source.

2.3 Modèle de Repository

Dans notre browser, nous avons également utilisé un modèle afin d'une manipulation plus facile à associer les persistance d'Hibernate et notre base de données. Ce modèle

est le plus populaire actuellement. Il nous suffit de créer un repository et une couche de service afin de le contrôler, tandis qu'il finira automatiquement le reste du travail. Son objectif principal était l'abstraction du code d'accès à la base de données. JPA fournit déjà cette abstraction. Il n'est donc pas nécessaire de créer une autre couche offrant les mêmes fonctionnalités.

Le modèle de Repository est assez simple. Une interface définit le repository avec toutes les opérations de lecture et d'écriture logiques pour une entité spécifique. L'interface est implémentée par une ou plusieurs classes fournissant des implémentations spécifiques aux bases de données pour chaque méthode d'interface. Notre Repository hérite qui issu d'un open source appelé « JPARepository ». Il fournit un ensemble de méthodes permettant de lire, de conserver, de mettre à jour et de supprimer une entité de base de données sous-jacent. Par exemple : enregistrer une nouvelle entité ou une entité modifiée, supprimer une entité, trouver une entité par sa clé primaire et trouver une entité par son titre. Nous pourrions lancer la recherche suivante : `findByConceptId`, `findBySenseID`, `findConceptByName`, etc., tandis que la librairie gère la recherche automatique.

2.4 L'implémentation de Browser

Dans un premier temps, nous créons un projet Spring boot. Dans ce dernier, la partie Hibernate est écrite principalement les persistance, ainsi un fichier "xxxService" afin de manipuler les services de tableaux, par exemple `ConceptService`, `WordService`, etc. Ce genre de fichier java contient principalement les fonctions utilisées pour les services de tableau, comme l'affichage du nombre de mots dans la page. Nos données étant assez volumineuses, nous limitons l'affichage à 30 éléments. Principalement, les fonctions réalisées dans ce browser sont les méthodes GET et POST, et les méthodes GET, POST, PUT, DELETE pour API. Étant donné que HTML ne supporte pas les méthodes PUT et DELETE, ainsi que la limite de temps, nous n'implémentons que ces deux fonctions GET et POST. Pour ces deux fonctions, ce browser permet de :

1. ajouter une unité de tous les tableaux. Par exemple : nous permettons l'ajout d'un mot à travers de remplir les features de la tableau "Word".
2. chercher les mots à travers ID et Lemma et les concepts à travers ID.
3. montrer les 30 premiers items dans notre base de données.

La première fonction permet d'ajouter les nouveaux composants dans la base de données. Quand L'information n'a pas été rempli par l'utilisateur. L'application informera l'utilisateur que "Information not complete!". Ajout d'un mot sera créer automatique une `wordForm` vide, `leffSem` vide et `Sense` vide. Tous les features seront remplis par "null" ou "0" selon les types de variable dans le tableau. Par exemple, si "origin" est une variable "String", la valeur de cette variable sera "null", si "neg" est une variable "Intéger" la valeur de cette variable sera rempli par "0", et ainsi de suite. Notons que la feature "word_id" sera crée automatiquement. Elle n'a pas besoin d'être créée par l'utilisateur. Dans cette fonction, tous les types de features ID, comme `Sense Id`, `concept Id`, `LeffSem Id`, seront initialisés par "java.io.Serializable". Dans le résultat de création de mot ou la recherche d'un mot, nous avons désigné que nous pourrions ajouter un sens,

un leffsem, un wordform de ce mot via un link en bas de chacun de ces tableaux. Ainsi, nous n'aurons pas besoin d'ajouter immédiatement lorsque nous créons un nouveau mot comme expliqué précédemment.

Create a Word :

Lemma :
ex : abandon
 Cpos :
ex : NOUN
 Pos :
ex : NC
 Language :
ex : fr
 Inflection :
ex : nc-2m
 Tag :
ex : NULL
 Modal : ☒ 0 ☐ 1
ex : 0
 Negation : ☒ 0 ☐ 1
ex : 1 is negation, 0 is adverb.
 Disabled : ☒ True ☐ False
ex : true is activated, false is adverse.
 Origin :
ex : lefff-3.4

[Home page](#)

(a) Créer un nouvel mot

Word

Id	cpos	pos	Lemma	lang	infl	tag	modal	neg	disabled	origin	update this word
237,781	Cpos	Pos	Nouvel_mot	fr	Infle	Tag	0	0	yse	lefff-3.4	237,781

Sense

ID	Word id	Score	concept	domain	origin	update this sense
434,239	237,781	0	9	0	null	434,239

[Add Sense of this word](#)

Word Form

id	word id	disabled	form	morph	lefffPos	update this word form
1	237,781	no	null	null	null	1

[Add Word Form of this word](#)

Leff Sem

id	form	pred	fram	tag	syntax	wordId	update this leffSem
1	null	null		null	null	237,781	1

[Add LeffSem of this word](#)

(b) Le résultat

FIGURE 2.3 – Une galerie de fonction de "Add Word"

Sachant qu'un mot a un sens, une forme, un leffSem dans notre dictionnaire et afin d'éviter de créer ces quatre tableaux (mot, sens, forme, leffsem) en même temps, nous avons établi une fonction qui se situe en bas de chaque tableau. "Add xxx of this word" permet d'ajouter le sens, le leffsem, la forme de ce mot, cette fonction link to chaque fonction de création. Par exemple, après avoir ajouté un nouveau mot dans notre base de données, nous voulions ajouter le sens de ce mot. En cliquant sur "Add Sens of this word", nous serons dirigés vers la page "Create a Sense : ". Après avoir créé ce sens, nous serons renvoyés vers le résultat de ce mot, comme décrit à la figure 2.3(b).

Create a Concept :

Score :
ex : 1
 Disabled :
ex : true or false
 name :
ex : eng-30-000001740-a
 Definition :
ex : (usually followed by to) having the necessary means or skill or know-how or authority to do something
 Usages :
ex : able to swim // she was able to program her computer // we were at last able to buy a car // able to get a grant for the project
 Origin :
ex : wolf-1.4b
 Semclass :
ex : capability

[Home page](#)

(a) Créer un nouvel concept

Concept

Id	Name	Def	Usages	Origin	Semclass	Disabled	Update this concept
129,245	fr-00-0000000-n	Une nouvelle définition	Une nouvelle usage	wolf-1.4b	Une nouvelle SemClass	yse	129,245

Word of Concept

Id and Name

[Link a word with this concept](#)

Relation with other concept

Relation

[Add Relation of this concept](#)

[Home page](#)

(b) Le résultat

FIGURE 2.4 – Une galerie de fonction "Add Concept"

Pour la création de concept, on sera pas créer automatiquement l'élément lié à tableau concept, dont "nombre de mot de ce concept" et "la relation avec autre concept", deux fonction linké en bas de ces deux tableaux comme ce qu'on a expliqué précédemment, on ne mentionne pas ici encore une fois. La fonction "Link a word with this concept" permet de lier un mot avec ce concept. Cette fonction n'a pas réalisé, mais on met un lien ici afin de noter la pensée. La fonction de "Add Relation of this concept" permet d'établir une relation entre deux tableaux. Notre tableau de "relation" contient les features : "source_id", "target_id", "name", "description", "domain_id", "disabled", "origin". Pour le feature "source_id" et "target_id", ces deux id peuvent être les mêmes mais ils ont des relations différentes. Ainsi, nous aurons besoin d'établir une relation bidirectionnelle via Hibernate.

Word

Id	cpos	pos	Lemma	lang	infl	tag	modal	neg	disabled	origin	update this word
114	ADV	ADV	à peine	fr	inv		o	o	no	lefff-3.4	114

Sense

ID	Word id	Score	concept	domain	origin	update this sense
278,688	114	4	18	1	wolf	278,688
278,694	114	4	26	1	wolf	278,694
280,105	114	4	1,426	1	wolf	280,105
281,977	114	4	3,763	1	wolf	281,977

[Add Sense of this word](#)

Word Form

id	word id	disabled	form	morph	lefffPos	update this word form
219	114	no	à peine		adv	219

[Add Word Form of this word](#)

Leff Sem

id	form	pred	fram	tag	syntax	wordId	update this leffSem
1,089,127	à peine	à peine_____1		Default	%default	114	1,089,127
1,089,128	à peine	à peine_____2		Default	%default	114	1,089,128

[Add LeffSem of this word](#)

Find word by Lemma

[Home page](#)

(a) Chercher le mot "à peine"

(b) Le résultat

FIGURE 2.5 – La fonction "Find word by Lemma"

Pour la deuxième fonction de ce browser, nous demandons en premier lieu une boîte de dialogue afin de récupérer le mot d'utilisateur voulais chercher. Dans la figure 2.5(b), nous mettons en avant tous les features qui concernent ce mot, par exemple : le sens de ce mot, sa forme, leffSem de ce mot. En bas de chacun de ces tableaux démontrés, nous établissons un lien qui permet d'ajouter directement le sens, la forme, le leffsem de ce mot. À droite de chaque ligne dans ces tableaux, nous établissons un lien afin de mettre à jour chaque sens, forme ou leffsem. Parfois, si nous recherchons un mot par lemma, il est possible de voir apparaître plusieurs mots, car ce mot a différentes formes

et leffsem etc. Ainsi, l’affichage de mot sera montré un par un.

Find concept by ID

Id:

[Home page](#)

Concept

Id	Name	Def	Usages	Origin	Semclass	Disabled	Update this concept
18	eng-30-00002621-b	only a very short time before	they could barely hear the speaker // we hardly knew them // just missed being hit // had scarcely rung the bell when the door flew open // would have scarce arrived before she would have found some excuse to leave (W.B.Yeats)	wolf-1.4b	SubjectiveAssessmentAttribute	no	18

Word of Concept

Id and Name
51.011 guère
114 à peine

[Link a word with this concept](#)

Relation with other concept

Relation
Add Relation of this concept

[Home page](#)

(a) Chercher le concept par id 18

(b) Le résultat

FIGURE 2.6 – Chercher un mot par ID

Une autre démonstration de cette fonction, recherche du concept par ID, l’affichage donnera tous les mots qui sont liés à ce concept. Nous pourrons également donner les instances de ce concept qui sera expliqué précisément dans le chapitre de matching d’ontologie. Finalement, pour la fonction de "Find xxx by xxx", si un mot ou un concept n’existe pas dans notre dictionnaire, l’application sera dirigée vers la page de "Not existe". Pour la fonction de "Create xxx :", si ce mot existe, l’application sera dirigée vers la page "This xxx existe".

Word Form List

549229 word forms in Database.

Id	Word Id	Disabled	Form	Morph	LefffPos
1	237766	false	null	null	null
2	1	false	!		poncts
3	2	false	!!		poncts
4	2	false	!!!		poncts
5	2	false	!!		poncts
6	2	false	!!!		poncts
7	3	false	!?		poncts
8	3	false	!?!?		poncts
9	3	false	?!		poncts
10	3	false	?!?!		poncts
11	3	false	!?		poncts
12	3	false	!?!?		poncts
13	3	false	?!		poncts
14	3	false	?!?!		poncts
15	4	false	"		ponctw
16	4	false	"		ponctw

(a) Une liste de WordForm limite à 30

Concept List

128744 concepts in Database.

Id	Name	Definition	Usages	SemClass	Origin	Disabled	Score
1	eng-30-00001740-a	(usually followed by 'to') having the necessary means or skill or know-how or authority to do something	able to swim // she was able to program her computer // we were at last able to buy a car // able to get a grant for the project	capability	wolf-1.4b	false	3
2	eng-30-00001740-b	without musical accompaniment	they performed a cappella	AcapellaMusic	wolf-1.4b	false	4
3	eng-30-00001740-n	concept formulant la catégorisation et l'identique des choses de notre environnement		Entity	wolf-1.4b	false	2
		draw air into,	I can breathe better when the				

(b) Une liste de Concept limite à 30

FIGURE 2.7 – Une galerie de fonction "Show" de Browser

La dernière fonction de ce browser est de donner le nombre de mots existants dans notre base de données. Ensuite, les 30 premiers items s’affichent dans un tableau qui

est basé sur moteur de font : FreeMarker. C'est le cas également pour tous les autres tableaux.

Pour conclure, étant donné que HTML ne supporte pas la méthode "PUT" et "DELETE", même si nous pouvons créer un filtre pour appliquer ces deux méthodes, nous n'avons pas suffisamment de temps. Nous appliquons donc simplement les fonctions liées à "GET" et "POST". Pour la partie de CSS, nous pouvons appliquer les centralisations de tableau et la police de mot pour rendre l'affichage le plus clair possible. Pour la contrôle d'information dans la fonction de "Add" et "Find", si les mots ou les concepts etc ne sont pas existents dans notre bases de données, browser sera avertir "This xxx doesn't exist", par exemple : "This Word doesn't exist!".

2.5 L'implémentation de REST API

Le système REST que (Roy Fielding, 2000)[11] a désigné contient six contraintes : client-serveur, sans état, avec mise en cache, en couches, avec code à la demande (facultative), interface uniforme. Étant donné que notre but est simplement la maintenance de dictionnaire et l'ajout d'un nouveau composant, ainsi, on n'a faire que les deux de ces 6 partie, tels que la partie URL et la partie HTTP. Notre dessin de URL issu du livre REST-API-Design-Rule[14], ce API contient quatre méthodes GET, POST, PUT et DELETE, ainsi, pour diviser avec l'URL de browser, nous avons ajouté un path /api avant les restes de path. Nous présenterons, dans cette partie, le dessin d'URL. Toutes les notions concernant le dessin d'REST API sont tirées du livre de.

Le dessin de ce REST API autour de quelques questions telles que :

- Quand faut-il nommer les segments de chemin d'URL avec des noms au pluriel ?
- Quelle méthode de requête doit être utilisée pour mettre à jour l'état des ressources ?
- Comment mapper des opérations non-CRUD à mes URL ?
- Quel est le code d'état de réponse HTTP approprié pour un scénario donné ?
- Comment puis-je gérer les versions des représentations d'état d'une ressource ?
- Comment structurer un lien hypertexte en JSON ?

En suivant ces questions, L'URL de ce API est dessiné comme suit : `http://localhost:8080/api/nom de tableau(pluriel si la quantité est plus d'une entité, singulier sinon)/nom de fonction(par exemple : id/lemma/, etc. car nous avons une méthode pour chercher les mots par id ou lemma et chercher le concept par id)/ id ou lemma dans ce champ.`

Pour deux fonctions, création et mise à jour, nous aurons besoin de remplir les informations dans la case "Body" en format JSON, sauf l'information de "id". Celle-ci sera créée automatiquement par Hibernate. Prenons un exemple de mot :

GET :

Find all words : `http://localhost:8080/api/words/`

Find a word by id 15 : `http://localhost:8080/api/word/id/15`

Find a word by lemma "inspirer" : `http://localhost:8080/api/word/lemma/inspirer`

Pour trouver tous les mots dans notre dictionnaire, nous utilisons le pluriel de nom de tableau, par exemple : words, concepts, rels, etc. La quantité de tableau "word" étant importante, le temps d'exécution estimé sera conséquent et l'application sera arrêtée à cause de la mémoire de l'ordinateur. Le type d'erreur sera comme suit :

`o.s.w.s.m.m.a.HttpEntityMethodProcessor : Writing [java.lang.OutOfMemoryError : Java heap space]`. Ainsi, nous ne conseillons pas d'exécuter cette fonction. C'est la raison pour laquelle l'affichage de tableau dans browser est limitée à 30 items. Ensuite, pour chercher un mot par id, exemple "id/15", il s'agit de trouver un mot qui a l'identifiant 15.

POST :

Add a word : `http://localhost:8080/api/word/add/`

And filling-in information necessary with JSON format in the Body.

Pour cette fonction, nous aurons besoin de remplir toutes les informations au format JSON dans le "Body" comme la figure xxx. Pour le dessin d'URL, car on voulait ajouter simplement un mot, ainsi, la path "/word" sera singulier. Pareil pour tous les autres tableaux.

PUT :

Update a word 237779 :

`http://localhost:8080/api/word/update/237779`

And filling-in information necessary with JSON format in the Body.

Pour cette fonction, nous avons également besoin de remplir les informations dans la case "Body", mais dans la path, nous avons besoin d'ajouter "/id de mot ce que nous voulons mis à jour" comme l'exemple un peu plus haut.

DELETE :

Delete a word 15 : `http://localhost:8080/api/word/delete/15`

Delete all words : `http://localhost:8080/api/words/delete/`

Nous déconseillons d'utiliser cette fonction en raison d'inattention. Mais j'ai quand même appliqué cette fonction en raison de l'intégrité de REST API.

Pour conclure, à travers l'implémentation de ce browser et API lexicon, j'ai bien compris que les éléments nécessaires pour un browser et API lexicon sont : les persis-tances, une série de contrôle d'information, les redirections entre chaque page web, etc. Pour le futur travail, car nous basons sur ce JPARepository, nous pourrions faire une pagination, car la démonstration de résultat sera trop longue pour une page surtout si nous trouvons plusieurs mots, ou concepts ou autre. Une pagination aidera la clarté de l'affichage.

Chapitre 3

État de l’art : La Mesure Similarité Sémantique en ontologie croisée

Dans ce chapitre, nous présenterons un état de l’art via les notions de la mesure sémantique ainsi que les notions de matching ontologie afin que les quatre approches soient davantage compréhensibles par tous. En outre, les notions ne sont pas toutes écrites dans ce corps de rapport, seulement les notions utilisées. Je mettrai les autres notions dans l’annexe 2 ainsi que l’état de l’art non pas pour des ontologies croisées. La raison pour laquelle j’ai mis cet état de l’art dans l’annexe 2 est que ce genre d’état de l’art aide à l’avenir à travailler par exemple sur l’évaluation de la qualité de l’intégration d’ontologie. Toutes les notions de la mesure sémantique sont issues du livre *Semantic Similarity from Natural Language and Ontology analysis* [20]. Toutes les notions de matching dans ce chapitre sont issues du livre *Ontology Matching* [18]. Les deux auteurs de ce livre présentent de manière très claire toutes les notions de matching ontologie que nous avons appliquées pour nos états de l’art et nos approches.

3.1 Matching Ontology

Selon les auteurs Jérôme Euzenat et Pavel Shvaiko, le matching ontologie peut employer des termes différents pour désigner des concepts similaires, et inversement, un même terme peut désigner parfois des concepts différents (Chalupsky 2000). Ceci est particulièrement déroutant puisque ces termes peuvent être utilisés pour décrire à la fois une action et son résultat. Dans cette sous-section, nous présenterons uniquement quelques définitions pour notre approche :

Le « matching »[1] est le processus de recherche de relations ou de correspondances entre des entités d’ontologies différentes.

L’« ontology integration »[1] est l’inclusion dans une ontologie d’une autre ontologie et d’assertions exprimant le lien entre ces ontologies, généralement sous la forme d’axiomes de pont. L’ontologie intégrée est supposée contenir la connaissance des deux ontologies initiales. Contrairement à la fusion, la première ontologie est inchangée, tandis que la seconde est modifiée.

L'« ontology reconciliation »[1] est un processus qui harmonise le contenu de deux ou plusieurs ontologies, nécessitant généralement des modifications d'un des deux côtés, voire des deux côtés (Hameed et al. 2004). Dans ce cas, on ne procède pas à la fusion des ontologies, mais on parle de co-évolution. La réconciliation des ontologies peut être effectuée dans le but de fusionner deux ontologies ou dans le but de les rendre indépendantes.

3.1.1 Type d'hétérogène

Dans cette sous-section, nous présenterons la notion d'hétérogénéité, car notre premier étape du matching ontologie a pour but de la réduire entre WordNet et MeSH. L'hétérogénéité ne réside pas seulement dans les différences entre les objectifs des applications selon lesquelles elles ont été conçues ou dans les formalismes d'expression dans lesquels les ontologies ont été codées. Il existe de nombreuses classifications différentes pour les types d'hétérogénéité (Ghidini et Giunchiglia 2004). Certains experts se concentrent sur les inadéquations (Klein 2001), tandis que d'autres mentionnent plutôt les niveaux d'interopérabilité (Euzenat 2001). Nous considérons ici les types d'hétérogénéité entre WordNet et MeSH :

3.1.1.1 Conceptuelle hétérogène

L'hétérogénéité conceptuelle, également appelée hétérogénéité sémantique chez Euzenat (2001) et déséquilibre logique chez Klein (2001), représente les différences dans la modélisation du même domaine d'intérêt. Cela peut être dû à l'utilisation d'axiomes différents (et parfois équivalents) pour la définition de concepts ou à l'utilisation de concepts totalement différents, par exemple la géométrie axiomatisée avec des points comme objets primitifs ou la géométrie axiomatisée avec des sphères comme primitives objets. Comme indiqué par Klein (2001) et Visser et al. (1998), il existe une différence entre le déséquilibre conceptualisation, qui repose sur les différences entre les concepts modélisés, et le déséquilibre explicitation, qui repose sur la manière dont ces concepts sont définis et sont exprimés. Visser et al. (1998) fournissent une classification précise de ces différences. Enfin, dans le contexte des différences conceptuelles, Benerecetti et al. (2001) identifient trois raisons importantes afin que celles-ci tiennent. Nous en discuterons ci-dessous :

- La couverture est différente lorsque deux ontologies décrivent des domaines différents, se chevauchant éventuellement, au même niveau de détail et d'un point de vue unique.

- La granularité est différente lorsque deux ontologies décrivent le même domaine selon la même perspective mais à des niveaux de détail différents.

- La différence de perspective, également appelée différence de portée (Chalupsky 2000), se produit lorsque deux ontologies décrivent le même domaine, au même niveau de détail, mais dans une perspective différente.

3.1.1.2 Sémiotique hétérogénéité

L'hétérogénéité sémiotique, également appelée hétérogénéité pragmatique (Bouquet et al. 2004a), concerne la façon dont les entités sont interprétées par les personnes. En effet, les entités qui ont exactement la même interprétation sémantique sont souvent interprétées par l'homme en fonction du contexte, par exemple, de la manière dont elles sont finalement utilisées. Ce type d'hétérogénéité est difficile à détecter par l'ordinateur et encore plus difficile à résoudre, car il est hors de portée. L'utilisation envisagée des entités ayant un impact important sur leur interprétation, celles qui ne sont pas destinées à être utilisées dans le même contexte sont souvent sujettes aux erreurs. Compte tenu de la compréhension limitée qu'un ordinateur peut avoir sur ces questions, nous ne traiterons pas ici de l'hétérogénéité sémiotique. Habituellement, plusieurs types d'hétérogénéité se produisent ensemble. La première étape d'application consiste à réduire les types d'hétérogénéité. Les techniques permettant de traiter ces types individuellement sont présentées au chapitre IV - Approche utilisée.

3.1.2 Technique Élémentaire utilisée pour Matching Ontologie

Les techniques, au niveau des éléments, considèrent les entités d'ontologie ou leurs instances de manière isolée par rapport à leurs relations avec d'autres entités ou leurs instances. Toutes ces techniques sont basées sur la représentation de graphe.

3.1.2.1 Techniques basées sur les chaînes d'identifiant

Cette technique est appliquée sur notre premier algorithme. Les techniques basées sur les chaînes sont souvent utilisées pour faire correspondre les noms et les descriptions de noms d'entités d'ontologie. Nous avons utilisé cette technique afin de trouver le même concept avant de calculer la similarité d'extension et d'intention afin de définir quels concepts peuvent être intégrés. Ces techniques considèrent les chaînes comme des séquences de lettres dans un alphabet. Elles se basent généralement sur l'intuition suivante : plus les chaînes sont similaires, plus elles sont susceptibles de dénoter les mêmes concepts.

Habituellement, les fonctions de distance associent une paire de chaînes à un nombre réel, de sorte qu'une valeur plus petite indique une plus grande similarité entre les chaînes. Parmi les exemples de techniques basées sur les chaînes largement utilisées dans les systèmes de mise en correspondance, citons les préfixes, les suffixes, les distances d'édition et la similarité n-gramme.

3.1.2.2 Techniques basées sur le langage

Cette technique utilise aussi pour notre premier algorithme, les techniques basées sur la langue considèrent les noms comme des mots dans une langue naturelle, par exemple l'anglais. Ils sont basés sur des techniques de traitement du langage naturel exploitant les propriétés morphologiques des mots d'entrée.

Habituellement, ils sont appliqués aux noms d'entités avant l'exécution de techniques basées sur des chaînes ou du lexique afin d'améliorer leurs résultats. Cependant, nous considérons ces techniques basées sur le langage comme une classe distincte de techniques d'appariement, car elles peuvent être étendues naturellement, par exemple dans un calcul de distance (en comparant les chaînes résultantes ou les ensembles de chaînes). Cette classe englobe la classe de ressources linguistiques de la première édition, qui couvrait, entre autres, l'utilisation de lexiques ou de thésaurus spécifiques à un domaine, pour faire correspondre les mots (dans ce cas, les noms d'entités d'ontologie sont considérés comme des mots d'une langue naturelle), en se basant sur les relations linguistiques comme les synonymes ou hyponymes.

3.1.2.3 Techniques basées sur les graphes

Les techniques basées sur les graphes sont des algorithmes de graphes qui considèrent les ontologies en entrée, y compris les schémas de base de données et les taxonomies, comme des graphes étiquetés. Habituellement, la comparaison des similitudes entre une paire de nœuds des deux ontologies repose sur l'analyse de leurs positions dans les graphiques. L'intuition derrière cela est que, si deux nœuds de deux ontologies sont similaires, leurs voisins doivent également être similaires. Outre les techniques purement basées sur les graphes, il existe d'autres techniques plus spécifiques basées sur la structure, telles que l'utilisation d'arbres.

Deux premiers techniques, on a principalement utilisé leur normalisation afin que notre premier algorithme soit un contrôle d'identifiant plus précis et plus correcte pour trouver le meilleur identifiant possible. Étant donné que les quatre approches présenteront dans la section "État de l'art" utilise également ces techniques, ainsi, nous présenterons ici les notions, et dans le chapitre IV - Approche utilisée, nous préciserons plus concrètement comment nous implémentons ces techniques dans notre premier algorithme. La dernière technique est utilisée principalement pour notre troisième algorithme. Dans la section suivante, nous présenterons les notions d'approche de similarité sémantique.

3.2 La Mesure Sémantique

La mesure sémantique (MS) vise à saisir la force de l'interaction sémantique entre les éléments (par exemple, les mots, les concepts) en ce qui concerne leur signification (Harispe Sebastien et.al 2015)[20]

Les mesures sémantiques sont des outils mathématiques permettant d'estimer quantitativement ou qualitativement la force de la relation sémantique entre des unités de langage, des concepts ou des instances au moyen d'une description numérique ou symbolique. Cette description est obtenue par la comparaison d'informations qui corroborent formellement ou implicitement leurs relations, tout en signifiant ou décrivant leur nature. (Harispe Sebastien et.al 2015)[20]

Il est important de souligner la diversité du domaine (au sens mathématique) sur

lequel les mesures sémantiques peuvent être utilisées, comme pour effectuer des comparaisons mot à mot, concept à concept, texte à texte ou même d'instance à instance.

Notre approche sera principalement basée sur la « similarité sémantique » entre les concepts généraux et les concepts médicaux, avec les relations taxonomies (IS-A dans MeSH, hyperonyme et hyponyme dans WordNet), en utilisant WordNet comme une ontologie de référence pour des concepts médicaux de MeSH.

3.2.1 La Sémantique Similarité

La similarité sémantique concerne le calcul de la similarité entre des concepts qui ne sont pas forcément semblables lexiquement. Il s'agit d'un problème important dans les recherches sur Natural Language Processing(NLP) et Information retrieval(IR) et a suscité une attention considérable de la part de la littérature. Plusieurs approches algorithmiques pour calculer la similarité sémantique ont été proposées. La détection de similarités entre concepts ou entités est possible si ceux-ci partagent des attributs communs ou s'ils sont liés à d'autres entités, elles-mêmes sémantiquement liées dans une ontologie. Pour relier des concepts dans différentes ontologies, la similarité sémantique consiste à découvrir des relations ou affinités linguistiques entre des termes ontologiques à travers différentes ontologies.

3.2.2 Quatre branches de Similarité Sémantiques

Les similarités sémantiques qui s'appuient sur la base de connaissances peuvent être utilisées pour comparer chaque paire de nœuds exprimée dans un graphe définissant un ordre (partiel). Ainsi, chaque graphe est structuré par des relations transitives, réflexives et antisymétriques, telles que l'IS-A, Part-of, etc. Les principales approches utilisées pour comparer les concepts définis dans une taxonomie sont les suivantes :

Les mesures basées sur l'analyse de la structure des graphes. Elles estiment la similarité en fonction du degré d'interconnexion entre les concepts et sont généralement considérées comme des mesures encadrées dans le modèle spatial. La similarité de deux concepts est estimée en fonction de leur distance dans le graphique, par exemple sur la base de l'analyse des longueurs des chemins qui relient les concepts. Nous pouvons également envisager que ces mesures sont encadrées dans le modèle transformationnel en les considérant comme des fonctions estimant la similarité de deux concepts en ce qui concerne la difficulté de transformer un concept en un autre.

Les mesures basées sur l'analyse des caractéristiques du concept. Cette approche extrait les caractéristiques des concepts du graphique. Ces dernières sont ensuite analysées pour estimer la similarité en fonction de particularités communes et distinctes des concepts comparés. Cette approche est conceptuellement encadrée dans le modèle d'entité. La diversité des mesures basées sur les caractéristiques repose sur la diversité des stratégies proposées pour déterminer les spécificités des concepts et en tirer parti pour évaluer la similarité.

Les mesures basées sur la théorie de l'information reposent sur une fonction utilisée pour estimer la quantité d'informations véhiculées par un concept, c'est-à-dire son In-

formation Content (IC). Ces mesures permettent d'évaluer la similarité entre la quantité d'informations partagées et distinctes entre les concepts comparés. Cette approche est encadrée dans la théorie de l'information. Cependant, dans certains cas, elle peut être considérée comme un dérivé de l'approche basée sur les fonctionnalités, dans laquelle ces dernières ne sont pas comparées à l'aide d'une évaluation d'appariement de fonctionnalités booléenne (partagées / non partagées), mais intègrent également leur clarté, c'est-à-dire leur degré d'informativité.

Les mesures hybrides s'appuient sur plusieurs paradigmes. Nous présentons ici les quatre types de mesure de la sémantique similarité. Dans la communauté de matching ontologie, presque 90% d'approche sont divisé leur approche par ces quatre manières.

3.3 État de l'art

Dans cet état de l'art, nous présenterons quatre approche pour un ensemble de méthodes hybrides basé sur les ontologies croisées. La recherche d'état de l'art était dans IEEE, Elsevier, Springer, etc. La présente approche de structure est une simple introduction à une présentation d'approche globale. Les autres méthodes basées sur la même ontologie seront présentées dans l'annexe2. Une présentation des autres méthodes basées sur la même ontologie est également importante à l'avenir, par exemple : une évaluation de la qualité de l'intégration d'ontologie.

Ces méthodes exploitent le fait que les concepts entités comparés peuvent être associés à des propriétés, par exemple, la définition de concept ou l'exemple de concepts (sous la forme d'attributs), tout en tenant compte du niveau de généralité ou de spécificité de chaque entité. Au sein de l'ontologie ainsi que leurs relations avec d'autres entités ou concepts. Les mesures de similarité sémantique peuvent être utilisées pour effectuer des tâches telles que la récupération des résultats dans les requêtes des utilisateurs, la représentation et la redondance des ressources extraites, ainsi que pour vérifier la cohérence des ontologies. De nombreuses mesures de similarité sémantique avec une variété de propriétés intéressantes ont été proposées dans ces quatres approche ou dans l'annexe2. Dans la partie suivante, nous présenterons des mesures de similarité suivies d'une brève discussion de leurs méthodes.

3.3.1 X-similarité [5]

X-Similarity repose sur la correspondance entre les synsets et les ensembles de description de termes. Les ensembles de description de termes contiennent des mots extraits en analysant les définitions de termes. Deux termes sont similaires si leurs synsets ou leurs ensembles de description ou les synsets des termes dans leur voisinage (par exemple, des termes plus spécifiques et plus généraux) sont similaires lexicalement. Ils proposent de remplacer l'équation 3.1 par une similarité d'ensemble. On note ici les processus afin de percevoir la raison pour laquelle ils ont obtenu un final équation de 3.2 et de 3.3.

$$S(a, b) = \frac{|A \cap B|}{|A \cup B|} \quad (3.1)$$

A et B désignent des synsets ou des ensembles de description de terme. Étant donné que tous les termes situés au voisinage d'un terme ne sont pas liés par la même relation, ils proposent que les similarités d'ensembles soient calculées par type de relation (par exemple, Is-A et Part-Of pour WordNet et uniquement Is-A pour MeSH).

$$S_{voisin}(a, b) = \max \frac{|Ai \cap Bi|}{|Ai \cup Bi|} \quad (3.2)$$

i dénote le type de relation. L'équation suggère de calculer la similarité entre les voisinages de termes, en faisant correspondre les relations de même type entre des synsets de termes plus spécifiques et plus généraux (c'est-à-dire que, pour chaque terme, l'union des synsets de tous les termes jusqu'à la racine de chaque hiérarchie de termes est prise en compte) et en prenant leur maximum. Les idées ci-dessus sont combinées en une seule formule comme suit.

$$Sim(a, b) = \begin{cases} 1, & \text{if } S_{synsets}(a, b) > 0; \\ \max\{S_{neighborhoods}(a, b), S_{descriptions}(a, b)\}, & \text{if } S_{synsets}(a, b) = 0. \end{cases} \quad (3.3)$$

$S_{descriptions}$ indique la correspondance des ensembles de description de terme. Les descriptions et les syntaxes sont calculées conformément à l'équation 3.3. Notez que deux termes avec au moins un terme synonyme commun sont similaires à 100 %.

Notre troisième algorithme base sur X-similarité, nous comparons également les voisinages concepts pour trouver les concepts intégrable. Néanmoins, on a un changement de cet algorithme qui pour notre troisième algorithme car les différents algorithmes implémentés. Nous présenterons précisément dans chapitre IV - Approche utilisée.

3.3.2 An Approach to Ontology Integration for Ontology Reuse (Enrico G. Caldarola, 2016)[4]

La raison pour laquelle nous avons choisi cet article est que nous avons également une réflexion d'intégration. Dans la partie d'intégration, nous avons utilisé une notion de concept recyclée, ainsi que cet article utilisé également à la fin. Sauf que la partie évaluation de cet article est demander les experts à faire. Les restes partie d'article semble beaucoup de notre approche.

Dans cet article, l'auteur présente une approche de la réutilisation des ontologies basée sur des techniques de matching hétérogène. Précisément, il s'agit d'une nouvelle approche pour l'intégration d'ontologies orientée vers la réutilisation des ontologies. L'approche proposée sera appliquée au domaine de la production alimentaire. L'auteur utilise une combinaison de techniques de correspondance existantes basée sur des connaissances des experts du domaine et certains oracles linguistiques génériques. Une approche mixte dans cette approche, manuelle et automatique, dans la réutilisation et l'intégration d'ontologies.

3.3.2.1 Méthodologie de Matching dans cet article

L'objectif de la méthode de matching proposée consiste à créer un alignement, c'est-à-dire un ensemble de correspondances entre les entités issues des modèles d'entrée et celles de l'ontologie cible. Dans ce travail, l'ontologie cible peut être envisagée comme l'objectif final de l'approche d'intégration, mais dès les premières phases de l'approche proposée, elle constitue également une sorte de proto-ontologie englobant le domaine des mots-clés, des définitions de termes et des significations associées au domaine de connaissances cible et aux exigences d'application sur lesquelles les experts du domaine s'accordent. L'alignement est utilisé comme base pour l'intégration des ontologies d'entrée dans une conceptualisation cohérente et globale du domaine à l'étude.

L'auteur définit que la correspondance c comme le tuple :

$$c = (e1, e2, r, v) \quad (3.4)$$

où $e1$ étant une entité de la première ontologie, $e2$ étant une entité de la deuxième ontologie à comparer, r étant une relation d'appariement et v étant une mesure de similarité entre les deux entités basée sur la relation d'appariement particulière. Chaque correspondance peut être exprimée dans un format différent, tel que RDF / OWL, XML ou texte. Dans ce travail, nous utilisons une représentation des alignements basée sur RDF / OWL. Chaque entité impliquée dans une opération d'appariement a un identifiant unique correspondant à l'URI de l'ontologie en entrée.

Dans cet article, trois techniques de matching ont été utilisées : une technique basée sur les chaînes, une technique basée sur la linguistique, une technique basée sur la linguistique étendue. Nous avons également utilisé la technique basée sur les chaînes, ainsi, nous présentons les détails dans le chapitre IV - Approche utilisée. Les deux autres techniques ne seront présentées que brièvement.

Le matching linguistique est chargé d'une analyse complète des termes utilisés dans les modèles d'entrée au niveau sémantique, en utilisant une base de données linguistique externe comme WordNet ou d'autres sources de connaissances spécifiques à un domaine comme des connaissances de base. La similarité des chaînes et la mesure de la similarité sémantique résultant de l'opération de matching contribuent à définir la relation sémantique entre les entités. Les relations sémantiques correspondent aux relations théoriques entre les classes d'ontologies : équivalence ($=$), disjonction ($\perp$), moins générale ($\leq$), plus générale ($\geq$) et corrélation de concept ($\cap$). Une méthode de seuil est utilisée pour établir le type de relation de la théorie des ensembles qui existe entre les entités.

Le matching linguistique étendu définit et met en oeuvre un méta-modèle pour la mise en correspondance d'ontologies en utilisant autant que possible une conceptualisation proche de la manière dont les concepts sont organisés et exprimés en langage humain.

Dans leur approche, il construit dans un premier lieu un Sementic Network(SN) en utilisant les dictionnaire basé sur WordNet ou d'autres ressources spécifiques issues

d'autres domaines. Ils ont défini cet SN comme un graphe. Ils ont défini la pertinence du modèle de référence comme la somme de ses poids de mots pertinents (*terms centralities*) :

$$SSG(v) = \sum_{i=1}^n \varpi(i) \quad (3.5)$$

où n est le nombre de termes du modèle. SSG ils utilisent l'approche décrite dans (M. Rinaldi, 2013)[?] pour calculer une valeur global grade (GG) pour chaque réseau sémantique lié à chaque segment Modèle de référence sélectionné. Le GG est donné par la somme des grade syntaxique-sémantique (SSG) et le grade sémantique (SG). La première contribution eux donne des informations sur le modèle analysé en tenant compte de la polysémie du terme, c'est-à-dire de la mesure de l'ambiguïté dans l'utilisation d'un mot, donc, avec une définition précise du rôle de la considération du terme générique dans le modèle. Ils appellent cette mesure *centrality of the term i* et ils le définissent comme : $\varpi(i) = \frac{1}{poly(i)}$ où $poly(i)$ est la polysémie (nombre de sens) de i .

Après la construction d'étape de SN, ils le comparent avec le modèle d'entrée sélectionnées chaînes lexicales. L'autre contribution (SG) repose sur une combinaison de la longueur du chemin (l) entre des paires de termes et de la profondeur (d) de leur sous-utilisateur (c'est-à-dire le premier ancêtre commun), exprimée en nombre de sauts. De plus, à chaque propriété linguistique représentée par des arcs entre les noeuds du SN, un poids est attribué afin d'exprimer la force de chaque relation. Ils soutiennent que toutes les propriétés n'ont pas la même force quand elles relient des concepts ou des mots (cette différence est liée à la nature de la propriété linguistique considérée). Les poids sont des nombres réels dans l'intervalle $[0,1]$. Leurs valeurs sont définies expérimentalement et validées par des experts du point de vue de la comparaison des forces. Ils introduisent la définition de grade sémantique (SG), qui étend une métrique :

$$SG(v) = \sum_{(w_i, w_j)} e^{-\alpha \cdot l(w_i, w_j)} \frac{e^{\beta \cdot d(w_i, w_j)} - e^{-\beta \cdot d(w_i, w_j)}}{e^{\beta \cdot d(w_i, w_j)} + e^{-\beta \cdot d(w_i, w_j)}} \quad (3.6)$$

où (w_i, w_j) est une paire de mots dans l'intersection entre le Direct Semantic Network(DSN) et la référence de modèle v $\alpha \geq 0$ et $\beta > 0$ sont deux paramètres d'échelle dont les valeurs ont été définies par des expériences.

Une fois qu'ils ont obtenu la note globale pour chaque réseau mantique, elles sont comparées à une valeur seuil qui sert de filtre pour les modèles de référence en entrée, ils donnent ainsi le modèle en entrée le plus pertinent au niveau linguistique. Finalement, ils ont appliqué cette approche dans le domaine de la production alimentaire. L'inconvénient de cette approche est que ce processus nécessite beaucoup de travail manuel, même lorsqu'il traite de modèles communs et formels. Cette exigence peut constituer un obstacle majeur à une adoption généralisée de la réutilisation des connaissances et peut constituer un écart technologique important que les chercheurs devront combler dans un avenir proche. De plus, il est utile de tester l'approche proposée par rapport à d'autres domaines de la connaissance afin d'évaluer son applicabilité dans différents contextes.

3.3.3 S-Match : an algorithm and an implementation of semantic matching (Giunchiglia, 2004)[6]

S-Match est un système que Giunchiglia et al. ont développé en prenant deux arbres et en calculant la relation sémantique la plus forte entre les concepts des deux noeuds. La version actuelle de S-Match est une nouvelle implémentation rationalisée du système CTXmatch avec quelques fonctionnalités supplémentaires. S-Match a également défini les relations sémantiques correspondant aux relations théoriques entre deux noeuds de différentes ontologies : équivalence ($=$), disjonction ($\perp$), moins générale ($\leq$), plus générale ($\geq$) et corrélation de concept ($\cap$).

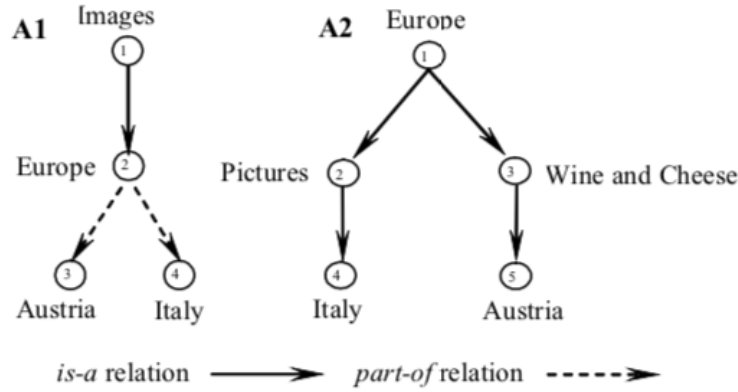


FIGURE 3.1 – Deux exemples d’hiérarchies concept dans S-Match

Les noeuds sont associés à un numéro et à une étiquette. Les numéros sont les identificateurs uniques des nœuds, tandis que les étiquettes sont utilisées pour identifier les concepts utiles à des fins de classification. Enfin, ils utilisent «C» pour les concepts de nœuds et d’étiquettes. Ainsi, CEurope et C2 en A1 sont respectivement la notion de label Europe et la notion de noeud 2 en A1. L’algorithme utilisé principalement dans S-Match est organisé en quatre macros phrases suivantes :

Étape 1 : pour toutes les étiquettes L des deux arbres, calculez CL ;

Étape 2 : pour tous les nœuds N des deux arbres, calculez CN ;

Étape 3 : pour toutes les paires d’étiquettes dans les deux arbres, calculez les relations entre CL ;

Étape 4 : pour toutes les paires de nœuds dans les deux arbres, calculez les relations entre CN.

Prenons un exemple tiré de l’article S-Match. Ils calculent d’abord CImages, CEurope, CPictures (étape 1) ; ils calculent ensuite C2 en A1 et C2 en A2 (étape 2) ; ils réalisent ensuite que CImages = CPictures (étape 3) et, enfin (étape 4), comparent C2 dans A1 à C2 dans A2 et, dans ce cas, ils réalisent qu’elles ont la même extension. Pour le détail de la mise en œuvre : les étapes 1 et 2 peuvent être effectuées une fois pour toutes, pour tous les arbres, indépendamment du problème de correspondance spécifique. Ils constituent une phase de prétraitement. Les étapes 3 et 4 ne peuvent être effectuées qu’au moment de l’exécution, une fois que les deux graphiques à faire

correspondre ont été choisis. L'étape 3 produit une matrice, appelée matrice CL , des relations entre les concepts d'étiquettes, tandis que l'étape 4 produit une matrice, appelée matrice CN , des relations les plus fortes entre les concepts de nœuds. Ces deux matrices constituent la sortie principale de leur algorithme de matching.

Dans leur algorithme, ils ont utilisé deux matrices principes, CL et CN , que nous avons noté un peu plus haut. Dans la matrice CL , ils ont défini deux éléments importants : "des machers au niveau des éléments de sémantique faible"¹ et "strong semantics element level machers"²

Dans la matrice CN , ils ont défini la formule suivante :

$$Context \rightarrow rel(C_i, C_j) \quad (3.7)$$

où C_i est le concept de nœud i du graphe 1, C_j est le concept de nœud j du graphe 2, rel est la relation sémantique (traduite de manière appropriée en une connotation propositionnelle) que ils veulent prouver comme étant la propriété entre C_i et, C_j , et $Context$ est la conjonction de toutes les relations (traduites de manière appropriée) entre les concepts de labels mentionnés dans C_i et, C_j .

L'idée clé est donc que le problème du matching a été traduit en un problème de validité dans lequel la tenue d'une relation sémantique entre les concepts de deux nœuds est testée en supposant que, en tant que *background theory* ($Context$ dans équation 3.7), tout ce qu'ils ont pu déduire de la relation tenant entre les concepts des étiquettes des deux graphes.

Ils ont ensuite implémenté une plateforme de sémantique matche en utilisant l'algorithme présenté précédemment. Son principal avantage est que l'algorithme qu'ils ont proposé est correct et complet dans le sens où il calcule toujours la relation sémantique la plus forte entre les concepts de nœuds (contrairement à ce qui se passe avec les coupleurs syntaxiques.). Ceci est en fait une conséquence évidente du fait que ils ont traduit le problème de la comptabilisation des concepts de nœuds (étape 4) en un problème de validité du calcul propositionnel. Cependant, il reste encore de nombreux problèmes qui rendent peu probable qu'ils puissent jamais construire le matcher sémantique « ultime pour tous ». S-Match repose sur deux fondements : premièrement, il utilise un framework de logique de prédicat pour unifier la correspondance chaîne-nœud avec la correspondance structurelle et l'ensemencement des connaissances. L'application directe de l'algorithme aux ontologies médicales est quelque peu faible en raison de sa dépendance au thésaurus WordNet.

Pour conclure en utilisant les notions de ce rapport, nous pourrions regrouper l'approche utilisant S-Match selon leur algorithme comme une approche hybride de la

1. Ces correspondants font essentiellement des manipulations de chaînes (préfixes, analyses postfixes, analyses n-grammes, distance d'édition, soundex, types de données, etc.) et tentent de deviner la relation sémantique implicitement codée dans des mots similaires.

2. Ces correspondants extraient les relations sémantiques existant entre les concepts d'étiquettes en utilisant des oracles qui mémorisent les connaissances nécessaires du lexique et du domaine (les oracles possibles sont, par exemple, WordNet, une ontologie de domaine, un thésaurus).

représentation de graphe et représentation de la base de connaissance telle qu’il a basé sur la méthode des caractéristique de concepts ainsi que la méthode basée sur l’analyse de la structure des graphes. L’algorithme utilisé principalement dans leur approche est la sémantique matching et les éléments choisis pour l’approche sont les sens et les tokens dans WordNet. Les techniques d’approche sont similaires aux techniques de l’ontologie Matching telles que la technique basée sur la langue comme ils ont utilisé la tokenisation et la lemmatisation, la technique basée sur les contraintes comme ils ont utilisé les sens et les tokens de WordNet pour construire un concept complexe¹, et aussi la technique basée sur le graphe comme il a utilisé aussi le concept père pour construire un concept complexe.

3.3.4 L’Approche variante de S-Match : Semantic Matching Using the UMLS(S-Match)(Shamdasani, 2009) [7]

Dans cet article, ils présentent un cadre de correspondance sémantique à utiliser dans le domaine médical. Le travail est une extension de l’approche S-Match. Ils décrivent les étapes nécessaires pour étendre l’algorithme S-Match au domaine médical pour une utilisation avec UMLS. S-Match est une approche de sémantique matching. Au lieu de définir des règles d’inférence, Giunchiglia et al. transforment le problème d’alignement d’ontologie en problème de satisfiabilité propositionnelle. Ici, les étiquettes des noeuds dans les arbres sont marquées et la structure lexicale, en plus de l’arborescence, est représentée par des formules logiques construites à partir de sens WordNet associés aux jetons. Ces formules sont introduites dans un solveur SAT standard pour vérifier la validité des correspondances en fonction de règles de raisonnement prédéfinies. L’un des points forts de cette approche est qu’il ne doit pas nécessairement y avoir un chevauchement lexical complet entre les ontologies à associer et WordNet.

Leur modification d’algorithme contient également quatre phrases qui correspondent à l’algorithme de S-Match. Leur algorithme modifié prend deux arbres en entrée et produit une matrice de relations entre eux. La figure 3.2 montre leurs arbres de saisie à cet effet. L’algorithme fonctionne sur le même principe, en quatre étapes, que l’algorithme d’origine. Au plus haut niveau de la première étape, l’étiquette d’un nœud dans une arborescence est convertie en une formule logique utilisant des règles spécifiques avec un filtrage effectué et des concepts attachés via une ressource en arrière-plan. L’étape 2 consiste à créer une formule de nœud et à appliquer un contexte sous forme de filtrage des concepts non pertinents. La troisième étape implique la mise en correspondance des formules atomiques à l’aide d’une ressource d’arrière-plan et la quatrième et dernière étape est le processus de raisonnement.

1. Lorsque les concepts atomiques existent, tous les jetons qui sont des prépositions, des marques de ponctuation, des conjonctions (ou des chaînes avec des rôles similaires) sont traduits en connecteurs logiques et utilisés pour construire des concepts complexes à partir des concepts atomiques construits à l’étape 3 ci-dessus. Ainsi, par exemple, les virgules et les conjonctions se traduisent par des disjonctions, des prépositions telles que *of*, *in* sont traduites par des conjonctions, etc. Par exemple, le concept d’étiquette *Vin et Fromage*, *CWine et Fromage* est composé de *CWine et Fromage* = $\langle \text{vin, sensWN } 4 \rangle \langle \text{fromage, sensWN } 4 \rangle$ où $\langle \text{fromage, sensWN } 4 \rangle$ est considéré comme l’union des quatre sens que WordNet attache au fromage, et de même pour le vin.

3.3.4.1 La modification d'algorithme de S-Match

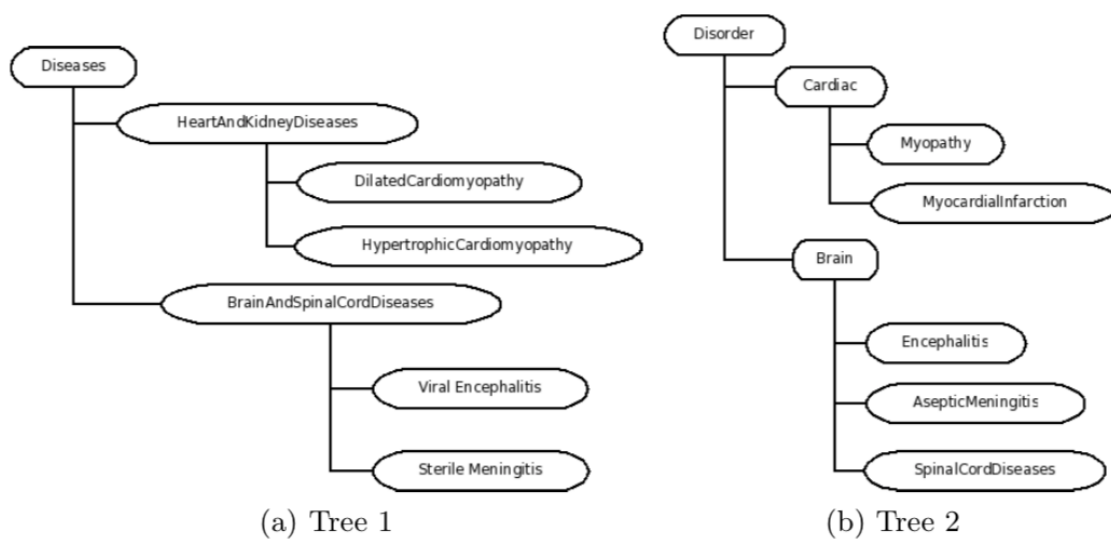


FIGURE 3.2 – Deux arbres entrée pour processus de matching

Leur méthodologie d'évaluation met l'accent sur la précision des différents schémas de matching et de filtrage hiérarchiques utilisés par leur algorithme. Ils ont sélectionné des sous-ensembles d'ontologies Foundational Model of Anatomy ontology(FMA¹) et MeSH pour leur processus de mise en correspondance et ont créé un alignement de références pour la tâche de mise en correspondance. Le sous-ensemble de la FMA choisi est basé sur le concept de cerveau et suit la relation de «partie régionale» avec ses nœuds feuilles, ce qui contient 476 concepts au total. Le sous-ensemble de MeSH choisi est également ancré dans le concept de cerveau et parcourt ses noeuds feuilles. Il contient 181 concepts au total. Ces deux sous-ensembles couvrent le même domaine, car ils décrivent les relations anatomiques avec l'organe « cerveau ». Toutefois, leur niveau de couverture, la dénomination des termes et les modèles sont différents. Toutes les correspondances vont de FMA à MeSH, c'est-à-dire que FMA est l'ontologie source et MeSH est la cible. Ils ont utilisé l'algorithme S-Match a été ré-implémenté à partir de zéro avec leur modifications intégrées et ils ont utilisé la bibliothèque SAT4J4² comme leur solutionneur SAT. Pour l'évaluation de leur algorithme, ils ont choisi d'utiliser différentes sources pour le filtrage des concepts (concepts, atomes ou co-occurrences, aucun) et pour la correspondance hiérarchique (concepts ou atomes). Ils ont créé leur alignement de références avec l'aide d'un expert médical. Leur processus de création de leur norme a consisté à sélectionner 20 concepts aléatoires dans leur sous-ensemble FMA

1. Le modèle fondamental d'ontologie de l'anatomie (FMA) est une source de connaissances en informatique évolutive pour l'informatique biomédicale, qui concerne la représentation des classes ou des types et les relations nécessaires à la représentation symbolique de la structure phénotypique du corps humain sous une forme qui : Compréhensible pour les humains, navigable, analysable et interprétable par des systèmes basés sur la machine, FMA est une ontologie de domaine qui représente un ensemble cohérent de connaissances déclaratives explicites sur l'anatomie humaine.

2. Sat4j est une bibliothèque JAVA pour la résolution de problèmes d'optimisation et de satisfaction booléenne.

et 40 concepts aléatoires dans leur sous-ensemble MeSH, et à en déduire manuellement une correspondance dans la plage d'appariement sémantique.

Dans ce travail, ils ont montré comment appliquer l'UMLS à une tâche de correspondance sémantique en modifiant l'algorithme S-Match pour son utilisation. Pour cela, ils ont détaillé les étapes à suivre et l'utilisation de l'UMLS. Ils ont constaté, en fonction du nombre de résultats montrés, que le filtrage avait un effet minimal sur le nombre de résultats découverts. Toutefois, dans certaines situations, ce n'est pas le cas. Globalement, la précision est élevée lorsque leur méthode est élevée, ce qui est démontré par les valeurs de haute précision de leur algorithme et le rappel est également élevé (les deux sont parfaits à 1), ce qui est à prévoir sur un si petit alignement de référence.

En conclusion, ces quatre articles sont ce que j'ai trouvé le plus récemment et leurs approches ont de nombreuses similarités avec notre approche. Nous pourrions également prendre ces approches comme la référence de notre approche. En particulier dans la dernière approche, une variante méthode qui est basée sur le système S-Match et ressemble beaucoup à notre but de matching. Ainsi, notre deuxième algorithme réfère à cette approche, pour deuxième approche "An Approach to Ontology Integration for Ontology Reuse". Cette approche est importante également pour notre réflexion d'intégration d'ontologie. Ainsi, après avoir établi un état de l'art, nous appliquons notre approche. Dans le prochain chapitre, nous présenterons notre approche en détail.

Chapitre 4

Approche utilisée

Dans ce chapitre, nous présenterons l’approche et les méthodes que nous avons utilisées ainsi que les résultats obtenus à chaque étape. Nous utilisons une approche hybride qui a choisi durant chaque étape d’implémentation de Baseline, selon différentes typologies rencontrées. Nous choisissons un des algorithmes correspondants qui pourra résoudre ce genre de typologie. Ainsi, la première étape d’implémentation consiste à trouver les concepts qui co-existent dans ces deux ontologies. Désormais, pour simplifier la présentation, nous supposons que le concept désigné par une étiquette est l’étiquette elle-même.

Pour la partie de matching, nous avons principalement utilisé les techniques suivantes qui sont présentes dans le livre *Matching Ontology* [18] :

1. Technique basée sur les chaînes ;
2. Technique basée sur la langue ;
3. Technique basée sur les graphes.

Pour rappel, étant donné que MeSH n’autorise pas une utilisation commerciale de ses données, nous n’avons pu les télécharger que dans un but de recherche. Ces données ont plutôt servi à résoudre la problématique soulevée dans le premier chapitre.

4.1 Les bases données utilisées

WordNet dispose d’un accès à python, ce qui nous permet d’accéder aux données WordNet rapidement et entièrement. Techniquement, WordNet est une ressource lexicale et il est nécessaire d’effectuer un travail supplémentaire pour la transformer en une ontologie formelle spécifiée dans une logique de formalisme (Gangemi et al., 2003a). Néanmoins, dans le but de matching et d’intégrer les concepts qui co-existent, la différence de définition n’est pas cruciale.

Étant donné que WordNet a un accès dans Python, nous pouvons extraire sept critères¹ dans WordNet et les stocker sous CSV selon nos besoins : `c_name` (identifiant

1. Les ontologies terminologiques (comme WordNet) ont des étiquettes de concepts (termes utilisés

du concept), synset (ensemble de synonymes), hyponymes (relation sémantique), hypernymes (relation sémantique), définition (définition du concept), exemple (exemple de concept), list_word(liste de mots).

Voici les raisons pour lesquelles j’ai extrait ces données : « c_name » correspond à l’identifiant du concept. Nous prenons ainsi la première instance (terme) des concepts (une liste de termes) telle que l’identifiant du concept pour la simple raison qu’il est plus facile de l’identifier après. « Synset » est un identifiant dans WordNet qui nous permet de faire des recherches si nécessaire. « Hyponymes » et « hypernymes » sont les relations sémantiques avec un ou plusieurs autres concepts qui permettent d’établir les relations. « Définition » est l’intention de concept dans WordNet, et enfin, exemple et « list_word » sont l’extension de concept ou les instances de concepts.

	c_name	synset	hyponymes	hypernyms	definition	exemple	list_word
0	name	synset	hyponymes	hypernyms	definition	exemple	list_word
1	entity	Synset('entity.n.01')	[Synset('abstraction.n.06'), Synset('physical_...)]	[Synset('entity.n.01')]	that which is perceived or known or inferred ...	[]	['entity']
2	physical_entity	Synset('physical_entity.n.01')	[Synset('causal_agent.n.01'), Synset('matter.n...)]	[Synset('entity.n.01')]	an entity that has physical existence	[]	['physical_entity']
3	abstraction	Synset('abstraction.n.06')	[Synset('attribute.n.02'), Synset('communicati...)]	[Synset('entity.n.01')]	a general concept formed by extracting common ...	[]	['abstraction', 'abstract_entity']
4	thing	Synset('thing.n.12')	[Synset('body_of_water.n.01'), Synset('inessen...)]	[Synset('physical_entity.n.01')]	a separate and self-contained entity	[]	['thing']

FIGURE 4.1 – Exemple d’extrait des concepts WordNet, 82 115 concepts extraits en total

Les descripteurs MeSH sont organisés dans un arbre MeSH, qui peut être considéré comme une hiérarchie de concepts MeSH. Dans l’arborescence MeSH, il existe quinze catégories (par exemple, la catégorie A pour les termes anatomiques), elles-mêmes divisées en sous-catégories. Pour chaque sous-catégorie, les descripteurs correspondants sont hiérarchisés du plus général au plus spécifique. Puisque les descripteurs apparaissent normalement à plus d’un endroit dans l’arbre, ils sont représentés sous la forme d’un graphique plutôt que celle d’un arbre. En plus de leur rôle d’ontologie, les descripteurs MeSH ont été utilisés à l’origine pour indexer les articles MEDLINE¹. Par exemple, un descripteur contient un ou plusieurs numéros d’arbre. Ainsi, « Temefos » contient trois numéros d’arbre : « D02.705.400.625.800 », « D02.705.539.345.800 » et « D02.886.300.692.800 ». Les termes descripteurs sont des concepts principaux ou des titres principaux. Les termes d’entrée sont les synonymes ou les termes associés aux descripteurs. Par exemple, le terme descripteur « Tumeurs » obtient les résultats suivants : « Cancers », « Tumeurs », « Tumeurs bénignes ».

pour les exprimer en langage naturel). Les relations lexicales entre les termes, par exemple la synonymie, l’antonymie, l’hypernyme et la métonymie, déterminent les positions relatives des concepts, mais ne les définissent pas complètement. La différence entre une ontologie terminologique et une ontologie formelle concerne son degré (Sowa, 2010). Si nous les représentons sous forme de graphique, une ontologie terminologique serait un type spécial de réseau sémantique.

1. est une base de données bibliographiques regroupant la littérature relative aux sciences biologiques et biomédicales.

Après avoir eu un aperçu de la structure de données MeSH, nous extrayons sept critères de concept parmi toutes ces catégories de MeSH et les stockons dans CSV. Les formats de concepts sont composés de : `c_ui` (identifiant unique du concept), `d_name` (nom de descripteur), `d_ui` (identifiant unique du descripteur), `rel` (relation avec autre concept), `scopeNote` (les notes de linguistique), `tree` (numéro d'arbre), `word_list` (liste de mots).

« `c_ui` » est l'identifiant unique du concept, ce qui nous permet de le rechercher rapidement. « `d_name` » est l'identifiant du descripteur de MeSH, qui se compose de deux langues, à savoir l'anglais et le français. Dans cette étude, nous avons matching les ontologies en anglais, et en avons donc extrait la partie de l'anglais. Ensuite, « `d_ui` » est l'identifiant unique du descripteur qui nous permet de rechercher dans l'arborescent de MeSH. Enfin, « `rel` » permet d'établir ou trouver une relation avec d'autres concepts. « `tree` » est composé d'un ou plusieurs numéros d'arbre de MeSH, qui permettent non seulement d'indexer ce concept, mais également de présenter une relation is-a. Ainsi, l'un des numéros de « Temefos » est « D02.705.400.625.800 ». Si nous retirons les trois derniers chiffres de cette chaîne, nous obtiendrions « D02.705.400.625 », qui correspond au descripteur précédent, « Organothiophosphates ». Le numéro d'arbre permet d'identifier le concept préféré d'une autre manière. Ce genre d'indexation d'arbre présente une relation similaire avec WordNet (hyperonyme et hyponyme). « `ScopeNote` » est principalement les descriptions textuelles des termes MeSH. Ce petit texte libre fournit un type de définition dans lequel la signification du MH est limitée. Il sera utile dans l'étape du calcul de la Sémantique Similarité. « `Word_list` » se compose d'une liste de mots/des instances de concept/des extensions de concept, et est également utilisé lors du calcul de la Sémantique Similarité.

	<code>c_ui</code>	<code>d_name</code>	<code>d_ui</code>	<code>rel</code>	<code>scopeNote</code>	<code>tree</code>	<code>word_list</code>
0	<code>c_ui</code>	<code>d_name</code>	<code>d_ui</code>	<code>rel</code>	<code>scopeNote</code>	<code>tree</code>	<code>word_list</code>
1	M0000001	Calcimycin	D000001	['NRW', 'M0353609', 'NRW', 'M0353609']	An ionophorous, polyether antibiotic from Stre...	['D03.633.100.221.173']	['T000002', 'Calcimycin', 'T000001', 'A-23187']
2	M0000002	Temefos	D000002	['NRW', 'M0352201', 'NRW', 'M0352200', 'NRW', ...]	An organothiophosphate insecticide.\n	['D02.705.400.625.800', 'D02.705.539.345.800', ...]	['T000008', 'Temefos', 'T000005', 'Abate', 'T0...
3	M0000003	Abattoirs	D000003	[]	Places where animals are slaughtered and dress...	['J01.576.423.200.700.100', 'J03.540.020']	['T000009', 'Abattoirs']
4	M0000004	Abbreviations as Topic	D000004	['NRW', 'M0511063', 'NRW', 'M0511063']	Works about shortened forms of written words o...	['L01.559.598.400.556.131']	['T698652', 'Abbreviations as Topic', 'T701041...

FIGURE 4.2 – Exemple d'extrait des concepts MeSH, il existe 29351 concept.

Finalement, j'ai extrait 29 351 concepts dans MeSH et 82 115 concepts dans WordNet. Les données de ces deux ontologies contiennent toutes les deux sept features qui ont la même nature pour chaque feature.

4.2 Le Matching d'Ontologie

Il nous semble pertinent de rappeler que la notion de matching correspond au processus de recherche de relations ou de correspondances entre des entités d'ontologies différentes. Ici, en ce qui concerne le choix d'entité, nous nous basons sur le concept. Pour rechercher une relation du même concept entre MeSH et WordNet, nous nous

demanderons comment définir deux concepts de différentes ontologies qui ont une relation de similarité. Comme nous l'avons présenté dans le chapitre précédent, il existe plusieurs approches, telles que l'approche basée sur le contenu d'entité, basée sur la structure d'ontologie, basée sur un ensemble de features entre deux entités et basée sur un mélange de méthodes.

Dans cette partie, nous cherchons à décrire la réalisation de toutes les approches que nous avons décrites auparavant et que nous avons appliquées en Python 3.2.7. Nous aborderons, par la suite, les bibliothèques telles Panda et DataFrame pour la lecture de données, et NLTK pour les traitements des chaînes de phrases, ainsi que la Gensim qui transforme les mots en vecteurs. etc.

Intuitivement et premièrement, nous utilisons l'identifiant de concept pour trouver le même concept dans deux ontologies différentes, et étant donné que nous utilisons l'identifiant comme terme pour se référer à ce concept, il existe alors forcément une similarité car les termes sont des expressions qui identifient des concepts. Ils sont donc souvent utilisés pour étiqueter des concepts dans des ontologies. En conséquence, la mise en correspondance d'ontologies pourrait présenter un avantage considérable en termes de reconnaissance et d'identification des chaînes. Ainsi, notre premier algorithme est l'égalité de chaîne.

Deuxièmement, ce qu'il existe probablement un problème pour algorithme Égalité de chaîne, c'est ce que lorsque l'identifiant d'un même concept est présenté dans deux ontologies de domaines différents. L'expert utilise un identifiant plus spécifique ou moins spécifique pour représenter ce concept dans son ontologie. Ainsi, il se référera à « Aspirin » dans le domaine général, tandis qu'il recherchera « Acetylsalicylic Acid » dans le domaine médical. Il est alors difficile d'identifier ce genre de concept par le simple biais de l'identifiant de concept. Nous aurons donc besoin d'une autre manière d'identification de concept. Ainsi, notre deuxième algorithme : la Mesure de la Similarité sémantique, sera basée sur l'extension et l'intention de concept. Ici, nous utilisons une méthode de similarité de phrases pour identifier le même concept dans deux domaines différents.

Pour une meilleure clarté, nous présenterons ici les notions d'extension et intention dans WordNet et MeSH. En logique, l'intension et l'extension sont deux façons de définir un concept. L'intention d'un concept correspond à sa définition, comme :

MeSH : « Abdomen » est un concept préféré, son intention est : « That portion of the body that lies between the THORAX and the PELVIS », son extension est : « Abdomen », « Ventre ».

WordNet : « abdomen » est un synset (concept), son intension est : « the region of the body of a vertebrate between the thorax and the pelvis », son extension est : « abdomen », « venter », « stomach », « belly ».

Si les intentions et extensions de deux concepts présentent une grande similarité, nous dirons que ces deux concepts sont le même concept. Notre but est d'intégrer la

partie générale de concept biomédical dans un concept général, afin d'enrichir notre dictionnaire. Il s'agit de l'une des raisons pour lesquelles nous avons choisi cet algorithme, car après une extraction de concept via cet algorithme, dans la partie d'intégration, il existe forcément une généralité pour chaque concept paires extrait à travers cet algorithme. La quantité de concepts présents dans ces deux ontologies est conséquente. En effet, nous recensons 29 351 concepts dans MeSH et 82 115 concepts dans WordNet. Aussi, l'implémentation de notre deuxième algorithme nécessiterait un temps d'exécution particulièrement long. Pour ce temps d'exécution réduit, nous implémentons respectivement un dictionnaire alphabétique.

Troisièmement, les interprétations diffèrent probablement selon les experts. Prenons le concept « Aspirin » :

la définition du concept « Aspirin » selon WordNet : *"the acetylated derivative of salicylic acid ; used as an analgesic anti-inflammatory drug (trade names Bayer, Empirin, and St. Joseph) usually taken in tablet form ; used as an antipyretic ; slows clotting of the blood by poisoning platelets"*

la définition du concept « Aspirin » selon le MeSH : *The prototypical analgesic used in the treatment of mild to moderate pain. It has anti-inflammatory and antipyretic properties and acts as an inhibitor of cyclooxygenase which results in the inhibition of the biosynthesis of prostaglandins. Aspirin also inhibits platelet aggregation and is used in the prevention of arterial and venous thrombosis. (From Martindale, The Extra Pharmacopoeia, 30th ed, p5)*

À travers ces deux définitions du même concept des différents domaines et afin d'affiner ce Matching, nous utilisons un troisième algorithme, basé sur la structure hiérarchique de graphe. Ce dernier algorithme sera une comparaison du premier ancêtre concept de deux ontologies. Ainsi, nous avons une paire de concepts ancêtres commun pour les concepts paires extraits par le deuxième algorithme. Prenons, par exemple, un concept « vegetarian.n.01 » dans WordNet et « Vegetarians » dans MeSH. Ce concept paire existe dans WordNet et MeSH, et leur premier ancêtre concept en commun qui est « person.n.01 » dans WordNet et « Persons » dans MeSH (Attention, premier ancêtre concept en commun n'est pas forcément premier concept père lié à ce concept paire). Ainsi, nous considérons que cette paire de concept « vegetarian.n.01 » et « Vegetarians » est intégrable définitivement. Néanmoins, quelle partie de ce concept MeSH sera intégrée dans quelle partie de ce concept WordNet ? Nous en discuterons dans la section « Intégration Ontologie ».

Ces algorithmes sont implémentés l'un après l'autre. En temps normal, il vaudrait mieux les implémenter parallèlement afin de savoir quelle ontologie dispose de plus de situations de concept, mais aussi afin de connaître quelle méthode est la meilleure pour effectuer un Matching de concepts entre MeSH et WordNet.

4.2.1 Premier algorithme : Égalité de chaîne

Dans un premier temps, nous présenterons la forme d’identifiant de concept de deux ontologies, ensuite, une normalisation pour les formes d’identifiants possible de traiter durant un temps limite, avant de décrire notre algorithme. Enfin, nous présenterons le résultat obtenu.

Dans WordNet, il existe deux formes :

1. Pour un mot simple, sa forme est originale et minuscule.
2. Pour un mot composé, WordNet utilise un tiret bas pour séparer les tokens.

Dans MeSH, les initiales de tous les identifiants sont en majuscule, tandis qu’elles sont en minuscule dans WordNet. Ainsi, il est nécessaire de modifier la casse des identifiants lorsque nous souhaitons passer de MeSH à WordNet. Les identifiants de concepts se composent de quatre types de formes :

1. Le premier type est un terme avec une forme originale qui peut être utilisée directement dans WordNet.
2. Le deuxième type est le terme avec une forme flexion.
3. Le troisième type correspond aux termes composés avec les virgules ; ce genre de terme nécessite un traitement et une définition concrète. Dans nos approches, nous avons utilisé le mot avant la première virgule et recherché le même concept dans WordNet. Après avoir eu un aperçu manuel, la plupart des termes avant les premières virgules correspondaient au concept hyperonyme de ce concept. Ainsi, nous pourrions utiliser ce genre de terme pour chercher son concept hyperonyme. S’il existe dans WordNet, nous considérons que ce concept peut aussi être intégré.
4. Le quatrième type correspond aux termes composés de chiffres et de mots spécifiques tels que « 2’,3’-Cyclic-Nucleotide Phosphodiesterases ».

Malheureusement, en raison du manque de temps, il nous est impossible de traiter toutes les formes d’identifiants de concepts. Cependant, il pourrait être intéressant de les étudier lors d’un futur travail. En effet, chaque forme d’identifiant de concept nécessite une analyse de données. Dans le but de finir une approche assez complète, nous avons décidé de traiter, dans un premier temps, les trois premières formes de MeSH, en ne transformant que les identifiants de MeSH, car cette version de Matching est basée sur WordNet. Dans les sous-sections suivantes, nous présenterons en détail les techniques ainsi que les méthodes employées. Bien qu’il existe de nombreuses autres méthodes, nous ne nous intéressons ici qu’aux méthodes que nous avons utilisées. Pour en savoir davantage sur les autres techniques, vous pouvez vous référer à l’annexe 2 - Les techniques élémentaires pour Matching ontologie.

4.2.1.1 Technique basée sur des chaînes et basées sur la langue

Avant de comparer les chaînes réelles qui ont une signification en langage naturel, les procédures de normalisation peuvent aider à améliorer les résultats des comparaisons ultérieures. Ici, nous avons combiné une normalisation de techniques basée sur les

chaînes, une normalisation de techniques basée sur la langue pour les deux premières formes et la normalisation basée sur les chaînes pour la troisième forme d'identifiant.

La normalisation de cas convertit chaque caractère alphabétique des chaînes en son équivalent en minuscule. Par exemple, CD devient cd et SciFi devient scifi. Néanmoins, dans WordNet, les identifiants n'étant pas trouvés avec une forme fléchie, nous utilisons alors également la forme originale pour nous assurer d'un résultat. Ces opérations de normalisation doivent être utilisées avec précaution pour plusieurs raisons.

Une méthode intrinsèque, la normalisation linguistique, il en existe plusieurs comme tokenisation, extraction de terme etc. Cependant, nous avons choisi ces deux normalisations : Lemmatisation et Stemmatisation. Une librairie a été créée en Python sous le nom de «nltk». Cette dernière permet d'effectuer automatiquement ces deux traitements.

Les méthodes basées sur des chaînes sont normalement tirées d'une partie de la structure de la chaîne (sous forme de séquence de lettres). Les méthodes basées sur des chaînes trouveront généralement les classes Book et Textbook similaires, mais pas les classes Book et Volume. Cependant, notre identifiant de concept ne s'applique pas sur ce cas normal. Il existe de nombreuses façons de comparer des chaînes en fonction de la manière dont elles sont visualisées : par exemple, une séquence exacte de lettres, une séquence de lettres erronée, un ensemble de lettres, un ensemble de mots. (Cohen et al, 2003b) comparent diverses techniques d'alignement des chaînes, allant des fonctions de type distance aux fonctions de distance basées sur des entités. Nous discuterons des méthodes les plus fréquemment utilisées.

Les méthodes basées sur la langue reposent sur l'utilisation de techniques de NLP pour extraire les termes importants d'un texte. La comparaison de ces termes et de leurs relations devrait permettre d'évaluer la similarité des entités d'ontologie qu'elles nomment et commentent. Bien que celles-ci reposent sur certaines connaissances linguistiques, nous distinguons les méthodes qui ne reposent que sur des algorithmes et celles qui utilisent des ressources externes telles que les dictionnaires.

4.2.1.2 Égalité des chaînes

Nous pourrions commencer notre algorithme Égalité de chaîne : L'égalité de chaîne est une similitude : $S \times S \rightarrow [0\ 1]$ telle que $\forall x, y \in S, (x, x) = 1$ et si $x = y, (x, y) = 0$. C'est-à-dire cet algorithme renvoie 0 si les chaînes considérées ne sont pas identiques et 1 si elles sont identiques. Cela peut être considéré comme une mesure de similarité. Cela peut aussi être effectué après une certaine normalisation syntaxique de la chaîne, par exemple une réduction, une conversion de codage, une normalisation des signes diacritiques.

Algorithm 1 Chercher les paires concepts intégrable en utilisant égalité de chaîne

Data: Une liste de concept MeSH

Result: Une liste de concept WordNet trouvés et une liste de concept MeSH extrait

Initialiser une liste pour stocker les concepts trouvé dans WordNet

Initialiser une liste pour stocker les concepts extrait de tous les concepts MeSH

while *Liste de concept MeSH ne termine pas* **do**

 Extraire l'identifiant à partir un concept MeSH

 Normalisation des chaînes de cet identifiant

 Normalisation de langue cet identifiant après la normalisation des chaînes

if *Forme d'originale d'identification MeSH ou celle après la normalisation égale à la normalisation d'identifiant de WordNet* **then**

 Utiliser le synset WordNet à trouver les sept features (un concept de WordNet et de MeSH contient sept features dans notre approche)

 Ajouter cet concept WordNet dans la liste de concept WordNet

 Ajouter cet concept MeSH dans la liste de concept MeSH

else

 Le prochain concept MeSH

end

end

Finalement, cet algorithme extrait 4 873 de paires concepts à partir de 29351 concept MeSH en total et 82 115 concept WordNet en total, ces paires concepts sont prêts pour notre deuxième algorithme.

4.2.2 Deuxième algorithme : Similarité sémantique

Le deuxième algorithme est basé sur une combinaison de méthodes d'apprentissage supervisées : Word2Vec Gensim + Smooth inverse frequency + cosinus similarité. Cette méthode est basée sur l'approche de la théorie de l'information. Nous utilisons la définition de concept (extension de concept) et liste de mots (intention de concept) à appliquer dans cette méthode. Nous employons une librairie « Gensim » afin de faire rapidement Word2Vec, mais allons tout de même présenter la théorie derrière cette technique(Word2Vec).

Cette combinaison de méthode décrit dans l'article de Adsieg¹, on l'utilise comme notre baseline de deuxième algorithme mais appliquer sur l'extension et intension de nos paires concepts extrait à partir de la première algorithme. Cet article a testé une liste de combinaison de méthode comme la suite :

1. Jaccard Similarity
2. Different embeddings + K-means
3. Different embeddings + Cosine Similarity
4. Word2Vec + Smooth Inverse Frequency + Cosine Similarity
5. Different embeddings + LSI + Cosine Similarity
6. Different embeddings + LDA + Jensen-Shannon distance

1. https://github.com/adsieg/text_similarity

7. Different embeddings + Word Mover Distance
8. Different embeddings + Variational Auto Encoder (VAE)
9. Different embeddings + Universal sentence encoder
10. Different embeddings + Siamese Manhattan LSTM
11. Knowledge-based Measures

La raison pour laquelle j'ai choisi *Word2Vec + Smooth Inverse Frequency(SIF) + Cosine Similarity* est que, dans cet article, celui qui est semblable à notre situation. Le temps de réalisation est le premier élément à prendre en considération durant le choix du deuxième algorithme. Ainsi que Word2Vec¹ est l'une des techniques les plus populaires d'apprentissage de l'intégration de mots à l'aide d'un réseau neuronal superficiel. Toutes les références de Adsieg issues de NLP Town, les articles concernent chaque traitement comme Différents embeddings vient de [19].

Ensuite, on présente brièvement la librairie « Gensim » : Différents types de modèles ont été proposés pour estimer les représentations continues des mots, notamment l'analyse bien connue par Analyse Sémantique Latente (ASL) et l'Allocation de Dirichlet Latente (ADL). Dans Gensim, il se concentre sur les représentations distribuées des mots appris par les réseaux neurones. ADL coûte cher en calcul sur des grands ensembles de données et ASL préserve les régularités linéaires entre les mots. Une représentation de mot Onehot peut entraîner une catastrophe de vecteur. Tous les modèles dans Gensim sont formés à l'algorithme du gradient stochastique⁴ et rétropropagation du gradient.

4.2.2.1 Prétraitement des données

Nous avons utilisé toutes les extensions et tous les intensions de concept MeSH et WordNet pour entraîner notre modèle Word2Vec, et éviter la perte des vecteurs de mots. Nous avons alors assigné, lors du contrôle de *Out Of Vocabulary*(OOV), un vecteur « random » au mot qui n'existe pas dans notre modèle, afin de limiter (voire d'éliminer) les bruits. Dans notre modèle, nous avons cherché un moyen de contrôler le vocabulaire qui n'existe pas.

Nous avons implémenté le code d'Adsieg mais avec notre intension et extension de concept, dans ce code. Dans un premier temps, il a utilisé "gensim.models.phrases". Cette méthode permet de détecter automatiquement les phrases courantes, expressions multi-mots, n-grammes de mots à partir d'un flux de phrases. Il a choisi d'ajouter les mots composés dans la liste de mots simples, par exemple ['New', 'York'] devient ['New York']. Si nous utilisons encore une fois ce code, nous pourrions changer ['New York'],

1. Il a été développé par Tomas Mikolov en 2013 chez Google[3]

2. Elle sert à découvrir de manière statistique la sémantique cachée et sous-jacente (latente) de mots dans un corpus de documents.

3. est un modèle génératif probabiliste permettant d'expliquer des ensembles d'observations, au moyen de groupes non observés, eux-mêmes définis par des similarités de données.

4. est une méthode de descente de gradient (itérative) utilisée pour la minimisation d'une fonction objectif qui est écrite comme une somme de fonctions différentiables.

5. est une méthode pour calculer le gradient de l'erreur pour chaque neurone d'un réseau de neurones, de la dernière couche vers la première.

'Times'] à ['New York Times']. Ce code qui regroupe les mots simples aux mots composés sera très utile pour l'accuracy d'entraînement. Ainsi, cela ne fonctionne qu'une seule fois, car intuitivement, deux mots de mots composés seront plus fréquents que trois mots composés ou plus, même si la quantité de trois mots composés ou plus va probablement dépasser la quantité de deux mots de mots composés. Pour entraîner le modèle Word2Vec Gensim, nous aurons également besoin de la fréquence de mot, des phrases ainsi que d'un nettoyage NLP comme par exemple éliminer les ponctuations, les mots les plus fréquents, les mots vides etc.

Après avoir utilisé la méthode "gensim.models.phrases", dans MeSH, nous avons collecté 507 493 mots bruits (unigram + bigrams) et 28 880 phrases. Dans WordNet, nous avons collecté 515 110 mots bruits (unigram + bigrams) et 73 965 phrases.

4.2.2.2 Entraînement des modèles

Après un prétraitement de données et une collection de mots unigram et bigram, nous commençons à entraîner le modèle avec le vocabulaire collecté. Dans un premier temps, nous créons un modèle vide en utilisant les paramètres par défaut de Gensim¹. Les plus importants paramètres considèrent comme algorithme entraîner, Nous avons choisi Skip-gram(SG) comme notre mode de formation, et le modèle Hierarchical Softmax(HS) comme notre modèle.

Dans « Gensim », définition de Skip-gram selon (Guthrie et al.) [23] : k-skip-n-grammes pour une phrase $w_1...w_m$ comme étant l'ensemble

$$\{w_{i1}, w_{i2}, ...w_{in} | \sum_{j=1}^n i_j - i_{j-1} < k\} \quad (4.1)$$

Les sauts-grammes signalés pour une certaine distance k permettent un total de k ou moins saute pour construire le n-gramme. Ainsi, les résultats « 4 sauts-n-grammes » incluent quatre sauts, trois sauts, deux sauts, un saut et zéro saut (n-grammes typiques formés à partir de mots adjacents). Dans Gensim, nous pourrions choisir « window (int, optional) » pour définir la distance maximale entre le mot actuel et le mot prédit dans une phrase, comme nos définitions de concept ne sont pas très longues comme un corpus. Ainsi, nous avons choisi 2-skip unigram + bigrams.

Après une construction de vocabulaire dans notre modèle vide en utilisant ces paramètres choisis, le temps de construire les vocabulaires de WordNet et de MeSH prend 0,1 seconde. Ainsi, nous avons obtenu le résultat : dans MeSH, Gensim a collecté 31 202 types de mots à partir d'un corpus de 507 493 mots bruits et 28 880 phrases, avec un paramètre "min_count = 20". Cela veut dire, ignorer tous les mots d'une fréquence inférieure à 20. Gensim a éliminé 88 342 mots bruits, et a gardé 419 151 mots (gardé 82

1. « Word2VecTrainables » est une des fonctions dans Gensim qui permette d'entraîner les modèles avec plusieurs paramètres de cette fonction. Cet objet représente le réseau neuronal interne peu profond utilisé pour entraîner les imbrications. La sémantique du réseau diffère légèrement entre les deux modes de formation disponibles (CBOW ou Skip-gram), mais vous pouvez le considérer comme un réseau de réseau avec une seule projection et une couche cachée que nous entraînons sur le corpus. Les poids sont ensuite utilisés comme incorporations (ce qui signifie que la taille de la couche masquée est égale au nombre d'entités self.size).

% des mots bruits). Avec un paramètre "sample = 6e-05 (cela veut dire 0.00006)", ce paramètre est le seuil pour configurer quels mots de fréquence plus élevée sont randomly downsampled. La plage utile est entre 0 et 1e-5. Nous avons utilisé la valeur par défaut dans le code de Adsieg car le résultat obtenu est satisfaisant. Ses données ressemblent aux nôtres et tester tous les paramètres prendrait un temps considérable. Avec ce paramètre, Gensim a downsampled 1 409 mots les plus communs et a éliminé 212 208 mots (50,6 % de mots à partir de 419 151 mots). Finalement, Gensim construit un arbre de Huffman (pour après Hierarchical Softmax) avec 3 634 de mots qui a une profondeur de noeud 14 et 300 dimensions.

Les paramètres sont les mêmes pour WordNet. Ainsi, pour le résultat, Gensim a collecté 73 965 types de mots à partir d'un corpus de 515 110 mots bruits et 28 880 phrases. Avec paramètre "min_count = 20", Gensim a éliminé 4 083 mots bruits, et a gardé 423 498 mots (82 % des mots bruits). Avec un paramètre "sample = 6e-05", Gensim a downsampled 1 535 mots les plus communs et a éliminé 233 223 mots (55,1 % des mots à partir de 423 498 mots). Finalement, Gensim construit un arbre de Huffman avec 4 083 mots d'une profondeur de noeud 14 et 300 dimensions.

Après ces deux modèles de vocabulaire construit dans notre modèle, nous commençons à entraîner notre modèle avec 3 634 mots de vocabulaire et 300 features dans MeSH et 4 083 mots de vocabulaire et 300 features. Ces deux modèles sont entraînés après 30 Epoch¹. Le modèle MeSH est entraîné avec 76 secondes, et 64 secondes pour WordNet. Avec ces deux modèles Word2Vec, nous pourrons enfin utiliser SIF et similarité cosinus pour le calcul final.

Pour calculer la similarité de phrase en utilisant ces deux modèles de Word2Vec, Adsieg a établi une classe "Sentence", pour construire une instance de "Sentence". Il contient quatre attributs : sentence_raw (la phrase n'a pas encore été traitée), normalized_sentence (la phrase après la normalisation), tokens_raw (une liste de mots extraite à partir d'une phrase normalisée), tokens_without_stop (une liste de mots élimine les stop-mots à partir la liste de mots bruits). Sauf que cette classe, Adsieg a établi encore une fonction pour éliminer le premier élément principal en utilisant Truncated singular value decomposition and latent semantic analysis(TruncatedSVD²).

Après la construction de classe "Sentence" et la fonction de l'élimination du premier élément principal, nous pouvons commencer à entrer "Sentence Embeddings". Ci-joint, un pseudo-code d'algorithme :

-
1. le nombre d'itération sur le corpus.
 2. Ce transformateur effectue une réduction de dimensionnalité linéaire au moyen d'une décomposition en valeurs singulières tronquées (SVD). En particulier, la SVD tronquée fonctionne sur les matrices nombre de termes / tf-idf, telles qu'elles sont renvoyées par les vectoriseurs dans `sklearn.feature_extraction.text.Danscecontexte, nous parlons d'analyses sémantique latente (LSA)`.

Algorithm 2 Similarité de phrase

Data: Deux listes phrases, deux dictionnaires de fréquence de mot, deux modèles de Word2Vec, un boolean de use_stop_word, le seuil de poids de mot

Result: Une liste de Similarité

Initialiser une liste afin de stocker embeddings de phrases

Initialiser une liste afin de stocker similarité de phrase

Sommer chaque dictionnaire des fréquences de mot

while *deux liste de phrases zippé ne terminent pas* **do**

 Tokeniser deux phrases dans chaque liste

 Éliminer les mots vides de ces tokens

if *le cas OOVs* **then**

 | Choisir un mot au hasard

else

 | Continuer

end

 Pondérer les mots selon SIF

 Embedding ces token à travers deux modèles entraîné

 Coller ces vecteurs de deux phrases dans la liste embedding

 Supprimer du composant commun à travers SIF

 Calculer la similarité en utilisant Similarité cosinus

 Coller la similarité de deux phrases dans la liste similarité

end

Smooth Inverse Frequency¹ prendre la moyenne du mot imbriqué dans une phrase a tendance à donner trop de poids aux mots qui ne sont pas pertinents du point de vue sémantique. Smooth Inverse Frequency tente de résoudre ce problème de deux manières :

Pondération : comme le baseline est tf-idf, SIF prend la moyenne pondérée du mot imbriqué dans la phrase. Chaque mot incorporé est pondéré par un $\frac{1}{(a + p(w))}$, où a est un paramètre généralement défini sur 0,001 et $p(w)$ est la fréquence estimée du mot dans un corpus de référence.

Suppression du composant commun : SIF calcule le composant principal des incorporations résultantes pour un ensemble de phrases. Il soustrait ensuite de cette phrase en intégrant leurs projections sur leur première composante principale. Cela devrait supprimer les variations liées à la fréquence et à la syntaxe qui sont moins pertinentes sémantiquement. En conséquence, SIF rétrograde les mots sans importance tels que mais, juste, etc., et conserve les informations qui contribuent le plus à la sémantique de la phrase.

Mathématiquement, la similarité des cosinus est une mesure de la similarité entre deux vecteurs non nuls d'un espace produit interne qui mesure le cosinus de l'angle entre eux. Le cosinus de 0° est égal à 1 et il est inférieur à 1 pour tout angle compris dans

1. Smooth inverse frequency ont été conçues à l'origine par [21] et le document correspondant a été présenté à l'ICLR 2017. Dans [13], cet article a fait une comparaison entre Baselines, Word Mover's Distance, Smooth Inverse Frequency et Pre-trained encoders.

l'intervalle $(0, \frac{\pi}{2}]$ radians. Il s'agit donc d'un jugement d'orientation et non de magnitude : deux vecteurs de même orientation ont une similarité cosinus de 1, deux vecteurs orientés à 90° l'un par rapport à l'autre présentent une similitude de 0, et deux vecteurs diamétralement opposés présentent une similitude de -1, indépendante de leur grandeur.

La similitude en cosinus est avantageuse car, même si les deux documents similaires sont très éloignés l'un de l'autre par la distance euclidienne (en raison de la taille du document), il est fort probable qu'ils soient encore plus proches l'un de l'autre. Plus l'angle est petit, plus la similarité du cosinus est élevée.

	Name_MeSH	def_MeSH	word_list_def_MeSH	Name_WN	def_WN	word_list_def_WN	Similarity_highest
0	Name_MeSH	def_MeSH	word_list_def_MeSH	Name_WN	def_WN	word_list_def_WN	Similarity_highest
1	Abattoirs	Places where animals are slaughtered and dress...	['places', 'animals', 'slaughtered', 'dressed'...]	abattoir	a building where animals are butchered	['building', 'animals', 'butchered']	[0.9999999999999998]
2	Abdomen	That portion of the body that lies between the...	['portion', 'body', 'lies', 'thorax', 'pelvis'...]	abdomen	the region of the body of a vertebrate between...	['region', 'body', 'vertebrate', 'thorax', 'pe...]	[1.0000000000000002]
3	Abetalipoproteinemia	An autosomal recessive disorder of lipid metab...	['autosomal', 'recessive', 'disorder', 'lipid'...]	abetalipoproteinemia	a rare inherited disorder of fat metabolism; c...	['rare', 'inherited', 'disorder', 'fat', 'meta...]	[0.9999999999999996]
4	Abomasum	The fourth stomach of ruminating animals. It l...	['fourth', 'stomach', 'ruminating', 'animals', '....]	abomasum	the fourth compartment of the stomach of a rum...	['fourth', 'compartment', 'stomach', 'ruminant...]	[1.0000000000000002]

FIGURE 4.3 – Exemple de concept extrait à partir de deuxième algorithme

Voici, une aperçu sur les concepts ce qu'on a extrait, on a pris deux phrases qui ont la plus haut de similarité. et les considère comme deux concepts intégrable selon cette deuxième algorithme. Finalement, on a extrait 3 019 paires concepts intégrable.

Pour la partie évaluation, nous utiliserons le troisième algorithme pour faire une évaluation de cette partie. Le troisième algorithme est basé sur la structure de graphe. Nous comparons les concepts trouvés via ces deux algorithmes. Si, par exemple, il existe des concepts trouvés par le troisième algorithme mais qui n'existent pas dans la liste de concepts trouvée avec le deuxième algorithme, nous définirons le rappel et la précision de cette manière.

4.2.3 Troisième algorithme : comparaison des concepts voisins

Dans cette section, nous utilisons une approche basée sur la structure de graphe. En ce qui concerne les concepts pouvant être intégré, nous n'utilisons que les concepts qui ont une relation directe avec le concept à comparer, à définir la similarité. Par exemple :

« Abdomen » est un concept intégrable, dont la relation est hyperonyme avec « body-part » et hyponyme avec « underbelly » dans WordNet.

« Abdomen » est un concept intégrable, dont la relation est hyperonyme avec « torso » et hyponyme avec « Abdominal wall », « Groin », « umbilicus », etc.

Dans nos calculs, nous ne tenons pas compte des autres relations. Ainsi, l'important est de savoir le pourcentage de concepts voisins entre deux ontologies. Dans notre cas, les concepts hyperonymes et hyponymes n'ont aucune similarité, aussi, nous ne intégrons pas ce concept d'« Abdomen ».

Dans cette étape, nous utilisons une Fuzzy pour calculer la similarité entre les identifiants de premier concept ancêtre commun de paire concept similaire qui est calculé par le deuxième algorithme, et pour extraire les premier concepts ancêtres commun.

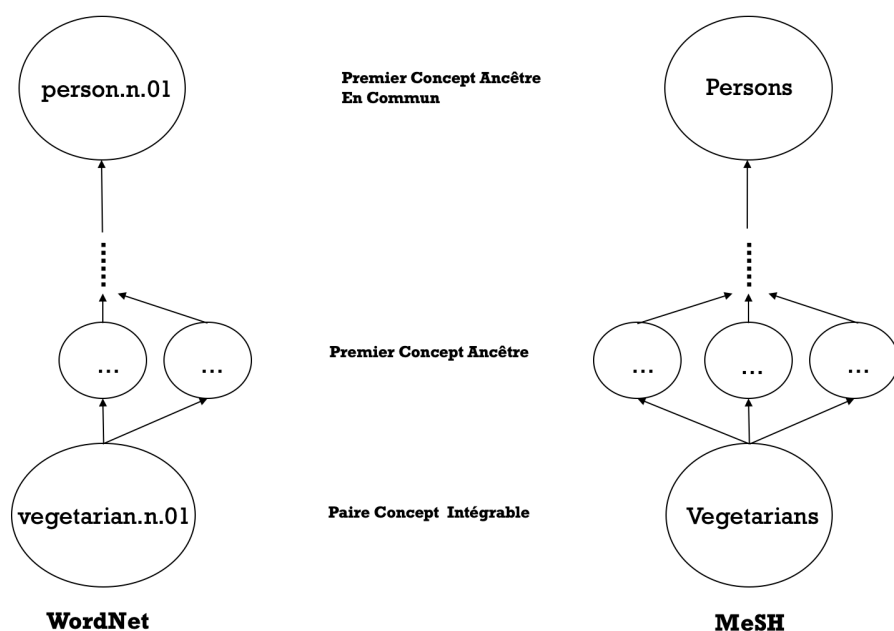


FIGURE 4.4 – Exemple de premier concept ancêtre en commun

Dans MeSH et WordNet, l'identique de père n'est pas simplement premier tranche de concepts père, c'est plutôt deux paires de père concepts qui existent dans deux ontologies. par exemple : une paire de concepts intégrables : "Vegetarians" dans MeSH et "vegetarian.n.01" dans WordNet, la premier tranche de père dans MeSH est "Persons [M01]", et dans WordNet, ce concept a un hypernyme concept comme : [Synset('eater.n.01')]. Si nous comparons seulement la première tranche de concept, cette paire de concept n'est pas intégrable.

Mais si nous cherchons les pères de ce paire de concept, nous trouverons "Persons [M01]". Ces deux concepts sont deux pères de concepts "Vegetarians" dans MeSH, et dans WordNet, nous avons également trouvé "[Synset('person.n.01')]".

Algorithm 3 Chercher la premier ancêtre concepts en commun

Data: une liste de paire concept, une liste de concept MeSH qui a les concepts hyponymes, une liste de concept WordNet qui a les concepts hyponymes

Result: Une liste de paires concepts trouvés

while *liste de paire concept ne termine pas* **do**

 Initialiser une liste pour stocker les paires concepts trouvés

 Extrait le tree number de child concept MeSH

 Chercher les premiers ancêtres concept MeSH en utilisant le tree number de child concept (appel d'une fonction récursif à chercher tous les ancêtres concepts de ce child concept)

 Stoker tous les premiers ancêtre concept MeSH dans une liste

while *une liste des premiers ancêtre concept ne terminent pas* **do**

 Extrait l'identifiant de premier ancêtre concept MeSH

 Extrait l'identifiant de premier ancêtre concept WordNet

if *La similarité entre l'identifiant de premier ancêtre MeSH et l'identifiant de premier ancêtre MeSH est supérieur ou égale à 80%* **then**

 Coller ces deux concepts, la similarité, deux identifiant de ce premier ancêtre concept commun dans la liste trouvé

else

 Comparer Le prochain premier ancêtre concept

end

end

end

Considérant les formes d'identifiant de concept MeSH et les formes d'identifiant de concept WordNet, ainsi qu'une vérification de résultat manuellement entre cinq seuils : 75%, 80%, 85%, 90% et 95%. On a définitivement pris 80% comme notre seuil de troisième algorithme. Finalement, on a trouvé 167 concepts intégrables parmi 717 concepts qui ont les concepts hyponymes.

4.3 Résultat de Matching

Ici, nous ne traitons que les deux premières formes d'identifiants de concepts MeSH avec les techniques basées sur la langue. Ainsi, après l'application de lemmatisation et de stemmatisation, l'identifiant de concept MeSH « Abdominal » deviendra « abdominal ». Dans Wordnet, en utilisant synset « Synset("abdomen.n.01") », nous avons obtenu 4 873 concepts avec ce genre de traitement.

Comme nous l'avons développé précédemment, deux concepts ne sont pas forcément identiques à travers leurs identifiant de concept. Ainsi, nous utilisons le deuxième algorithme pour trier ces co-concepts trouvés. Après avoir fini d'implémenter cette méthode, nous avons extrait 3 019 concepts qui peuvent être intégrés, y compris les concepts qui ont les identifiants des trois dernières formes.

De ces concepts co-existants, 2 301 concepts n'ont pas de concept hyponyme et pourront donc être intégrés directement.

Les 717 concepts restants nécessitent l'utilisation d'un troisième algorithme (technique basée sur graphe/structure). Nous avons ainsi obtenu 167 concepts intégrables parmi 717 concepts qui ont les concepts hyponymes.

Pour la méthode basée sur les chaînes, nous appliquons aussi sur la troisième forme de MeSH (celle avec les virgules). Après une analyse de ce genre de forme d'identifiant avec l'arbre de MeSH, nous avons découvert que le mot avant la première virgule correspond souvent au concept hyperonyme de ce concept. Ainsi, nous utilisons le mot avant la première virgule. Dans le cas de :« Diet, Vegetarian », nous recherchons le concept hyperonyme (qui peut également être intégré) de « Diet » si celui-ci existe dans WordNet. Nous avons ainsi trouvé 1 698 concepts co-existants. Néanmoins, les mots avant la première virgule ne figurent pas tous dans MeSH, aussi avons-nous également décidé d'utiliser les numéros d'arbre pour effectuer ce traitement. En fin de compte, la combinaison des méthodes a permis de trouver 789 concepts qui co-existent dans WordNet et MeSH.

Pour la dernière forme d'identifiant de MeSH, nous avons relevé 44 cas. Nous les avons regroupés dans une typologie que nous présenterons ultérieurement.

Finalement, nous considérons 2 468 ($2\,301 + 167$) concepts MeSH intégrables dans WordNet et ceux qui attendent une série d'algorithme pour déterminer quelle partie de ces concepts peut être intégrée.

4.4 Évaluation de Matching

Dans notre approche, nous utilisons le dernier algorithme pour une évaluation des résultats de deux algorithmes précédents. Comme nous avons utilisé le deuxième algorithme qui détermine 3 019 paires de concepts sont identiques, si les paires de concepts de troisième algorithme n'existent pas dans ces 3 019 concepts, cela veut dire que le premier et le deuxième algorithme ne sont pas assez. Ainsi, on a trouvé 50 paire de père concepts parmi des 167 concepts n'existe pas dans 3019 concept.

Après avoir fini les recherches sur les concepts intégrables via trois algorithmes (basés sur les morphologies, sur la sémantique et sur la structure d'ontologie), nous pourrions utiliser toutes ces features sur une réflexion d'intégration de concept de la partie générale. Par exemple : nous avons utilisé les définitions de concepts à faire un Matching, puis nous pourrions également utiliser les définitions de concepts à trouver dans la partie générale dans un concept. Ces trois algorithmes filtrent ou définissent les concepts intégrables. Ainsi, tous ces concepts intégrables contiennent toutes les features nécessaires à être intégrables. Par exemple : les features sémantique : l'extension et l'intension de concept, les features lexicale : l'identifiant de terme, les features structurelle : la comparaison de concept père.

4.5 Réflexion de l' intégration

Dans cette partie, nous voulions trouver la partie générale dans les concepts intégrables MeSH selon trois algorithmes de Matching. L'approche utilisée est la "Approaches to Computational Concept Creation" (Xiao et al., 2019). Cette approche est dans un contexte de (Boden, 1990). Plusieurs types de notions sont présentes dans cette approche, on retire simplement une de ces notions : Concept Recycling.

— "Concept Recycling" est la réutilisation créative de concepts existants (Aamodt et Plaza [1994]). (Les concepts de base restent généralement les mêmes ; le recyclage n'implique pas qu'ils doivent être supprimés.) Nous mentionnons deux méthodes typiques.

— Concept Mutation : modifie un concept en ajoutant, en supprimant ou en modifiant quelque chose. Par exemple, le concept de « téléphone mobile » a été modifié pour devenir « smartphone » en permettant l'utilisation de nouvelles applications sur les téléphones. Le changement de concepts existants est une forme bien connue de création de concept également avec l'homme ; Osborn [1953] énumère diverses techniques à cette fin. La généralisation et la spécialisation des concepts existants sont deux opérations de communication courantes.

— Concept Combination : combinez deux ou plusieurs concepts existants pour en former un nouveau, comme dans le cadre d'un mélange conceptuel, où « programme informatique » et « virus » peuvent former un mélange conceptuel de « virus informatique ». La mutation et la (re)combinaison de concepts sont largement utilisées dans les méthodes évolutives de création de concepts. Bien que le recyclage puisse sembler un moyen facile de créer de nouveaux concepts, un problème clé consiste à mesurer le sens et l'intérêt des concepts générés.

Avec notre situation, nous prenons plutôt la notion de "Concept Mutation", étant donné que nous voulons garder le concept WordNet en ajoutant la partie générale de concept MeSH et en changeant une partie de concept WordNet pour la cohérence de définition de concept. En règle générale, des termes plus généraux existent plus haut dans la hiérarchie et représentent un ensemble de nœuds plus petit que le plus grand nombre de termes plus spécifiques qui peuplent un espace beaucoup plus dense situé plus bas dans la hiérarchie. (Angelos Hliaoutakis, 2005). Avec cette notion d'ontologie générale, les connaissances générales sont applicable dans de nombreux domaines techniques. Cela veut dire leur concept sont utilisable dans nombreux domaines techniques. Ici, on prend quelque exemple afin qu'une réflexion d'intégration d'ontologie. Par exemple, Le système SENSUS (Swartout et al., 1996) fournit un moyen de construire une ontologie spécifique à un domaine à partir d'ontologies de sens commun données. L'idée de base est d'utiliser des éléments de départ, qui représentent les concepts de domaine les plus importants pour identifier les parties pertinentes d'une ontologie de niveau supérieur. Les parties sélectionnées sont ensuite utilisées comme points de départ pour étendre l'ontologie avec d'autres concepts spécifiques à un domaine. Le projet SKC (Composition de connaissances évolutive) (Jannink et al., 1998) vise à développer une algèbre permettant de composer systématiquement des ontologies à partir d'acto-

logies déjà existantes. Il offrira l'union, l'intersection et la différence comme opérations de base pour une telle algèbre.

En règle générale, des termes plus généraux existent plus haut dans la hiérarchie et représentent un ensemble de nœuds plus petit que le plus grand nombre de termes plus spécifiques qui peuplent un espace beaucoup plus dense situé plus bas dans la hiérarchie (Angelos Hliaoutakis, 2005). Plusieurs modifications ont donc été proposées pour rompre cette appréciation uniforme contraignante des arêtes. Des modèles implicites ou explicites définissant la force non uniforme de la connotation entre les concepts ont donc été introduits, par exemple (Young Whan et Kim, 1990 ; Sussna, 1993 ; Richardson et al., 1994).

Dans WordNet aussi, des concepts plus généraux (plus élevés dans la hiérarchie) avec de nombreux hyponymes donnent moins de contenu d'information que des termes plus spécifiques (plus bas dans la hiérarchie) comportant moins d'hyponymes. Prenons un exemple, dans WordNet, « Synset('playing.n.02') », ce concept a les concepts hyponymes : Synset('bowling.n.03'), Synset('catching.n.01'), Synset('golfing.n.01'), Synset('pitching.n.01'), et ce concept a un concept hyperonyme : Synset('action.n.01'), également pour MeSH. Cette approche est indépendante du corpus et garantit également que le contenu informationnel de chaque terme est inférieur au contenu informationnel de ses termes subsumés.

Ainsi, si nous voulions intégrer la partie générale de concept MeSH dans WordNet, nous devrions définir le poids au niveau de la taxonomie d'arbre de concept. Nous considérons que le concept, dans différentes positions, a une différence de généralité. Le poids de généralité dépend non seulement de la définition de concept mais aussi de la profondeur de ce concept. Ainsi, le quatrième algorithme sera basé sur la profondeur de concept et le contenu d'information de concept.

4.6 Évaluation via Recherche d'information

Finalement, pour la partie d'évaluation de la qualité d'intégration, après avoir discuté avec mon tuteur et mon encadrement, nous avons convenu de deux pistes réalisables :

1) La première piste d'évaluation semi-automatique serait d'entraîner sous supervision un modèle qui contient deux classes (positive et négative). La classe positive décrit les concepts « corrects » et sera entraînée par WordNet, étant donné que notre dictionnaire se base sur celui-ci. La classe négative définit les concepts « incorrects » et sera entraînée par tous les concepts de MeSH. Par la suite, les concepts co-existants sont intégrés, et de ce modèle, nous pouvons classer les concepts « corrects » « incorrects ». Il est important de noter que, pour un entraînement supervisé, la présence d'au moins un expert médical est obligatoire pour la partie annotation. Cependant, n'en disposant d'aucun, nous considérons que tous les concepts sont déjà annotés ou détiennent une annotation linguistique.

2) La deuxième piste d'évaluation automatique serait d'utiliser une liste de Q&A médicale, qui contient les termes de concepts dans leurs chaînes. Nous entraînerions deux modèles de Word2Vec : l'un d'eux serait entraîné par les vecteurs de WordNet, et l'autre, par les vecteurs de WordNet et MeSH (c'est-à-dire les concepts intégrés).

Pour la deuxième piste d'évaluation, nous avons trouvé des données du domaine médical : « HealthTap ». « HealthTap » contient 1 605 232 question et réponses qui permettent de réaliser notre deuxième piste. Avec ces deux modèles, nous pourrions obtenir deux similarités de phrases différentes. Une analyse de ces deux genres de similarités pourra nous aider à évaluer la qualité de l'intégration.

4.7 Conclusion

Nous n'avons pas réalisé ce quatrième algorithme, l'intégration et l'évaluation de la qualité d'intégration. Toutefois, nous en prenons note dans ce rapport. Après avoir fini une réflexion de l'intégration, notons ici notre pensée afin d'une intégrité de Baseline. Le post-traitement sera : 1) Affiner l'algorithme, 2) Transformer le format WordNet à notre base de données, 3) Activer les concepts selon le domaine.

Pour 1) nous pourrions utiliser les algorithmes qui existent dans état de l'art, par exemple : comptage d'étage.

Pour 2) dans notre base de données, nous avons appliqué la forme de WOLF, et un peu différent avec la forme de WordNet.

Pour 3) après avoir réalisé une transformation de format, nous avons deux champs qui se nomment "active" et "domain". Ces deux champs n'ont pas encore été dessinés, mais, nous en prenons note pour un futur travail.

Dans notre chatbot automate, nous ne pouvons pas activer tous les concepts pour différents domaines. Par exemple, dans le domaine général, nous n'avons pas "acetylsalicylic acid", concept dont les personnes ne sont pas familières si elles ne connaissent pas le milieu du biomédical. En général, il est plutôt appelé "Aspirin". De même, nous avons aussi des concepts qui se croisent de différents domaines. Par exemple, dans le domaine d'assurance, nous avons plusieurs types de contrats différents : contrat maladie, contrat juridique, etc etc. Ainsi, nous ne pouvons pas activer simplement les concepts selon les différents domaines, et aussi pour plusieurs domaines.

Chapitre 5

Typologie

Au fur et à mesure que nous avons implémenté notre approche, nous avons découvert sept typologies. Dans un premier temps, nous décrirons dans les sections suivantes, la situation dans laquelle nous avons rencontré ces typologies, avant de présenter une situation précise, et enfin, de proposer une solution d'algorithme.

5.1 Typologie 1 – Concept MeSH comme instance dans WordNet

Étant donné que notre premier algorithme utilise l'identifiant de concept MeSH pour chercher les concepts WordNet, ainsi, l'interface de WordNet affichera le synset une fois si cet identifiant est un synonyme de ce synset. Ce genre de concept qui ont 481 concepts, par exemple, « Abreaction » est un concept qui existe dans MeSH, mais pas dans WordNet. Néanmoins, ce concept est trouvé par l'interface de WordNet. Il s'apparente donc à une instance dans WordNet au lieu d'un concept. Comment peut-on, dans ce cas, définir cette typologie ? Nous proposons donc un deuxième algorithme pour résoudre ce genre de typologie.

5.2 Typologie 2 – Concept MeSH et WordNet ont différent concept hyperonyme

Nous avons également rencontré la deuxième typologie lors du deuxième algorithme. Nous avons utilisé une intention et une extension pour trouver les concepts similaires au niveau de la sémantique. Cependant, nous avons également découvert la deuxième typologie :

Deux concepts sont trouvés dans deux ontologies, mais avec différents concepts hyponymes et hypernyme. Nous avons relevé 789 cas qui présentent ce genre de problèmes, aussi, nous proposons un troisième algorithme afin de le résoudre.

5.3 Typologie 3 - Concept a hyponyme

Problématique ici après une réflexion sur la Mutation de Concept. : si nous intégrons tous les concepts sous arborescents des concepts intégrables MeSH dans WordNet, est-ce que ce concept intégrable WordNet reste encore un concept dans le domaine général ? Normalement, la réponse est négative. Prenons un exemple d'humain : si nous avons une paire de concept "Apnea" dans MeSH et "apnea.n.01" sont une paire de concepts considérés intégrables selon deux algorithmes précédents, et les sous-concepts de ce concept intégrable MeSH sont : ["Sleep Apnea Syndromes", "Sleep Apnea, Central", "Sleep Apnea, Obstructive", "Obesity Hypoventilation Syndrome"]. Les sous-concepts de ce concept intégrable WordNet sont : [Synset('periodic_apnea_of_the_newborn.n.01'), Synset('sleep_apnea.n.01')].

Si nous intégrons tous les sous-concepts de "Apnea" MeSH dans les concepts sous-arborescents "apnea.n.01" WordNet, le concept "apnea.n.01" WordNet reste-t-il encore un concept dans le domaine général ? Normalement, non. Les deux des features les plus importants d'un concept, ceux qui est non seulement les intentions et extensions, ceux qui est importante aussi, qui sont les concepts voisins.

Les concepts qui ont les hyponymes ont un problème d'intégration dans la partie générale, comme nous l'avons indiqué auparavant. Nous ne pouvons pas considérer simplement les paires de concepts comme les noeuds isolés. Ainsi, nous avons utilisé un simple Baseline pour filtrer les concepts ont les hyponymes concepts dans WordNet et nous avons trouvé 717 concepts qui ont les concepts hyponymes et ces concepts attendent une vérification via le troisième algorithme.

5.4 Typologie 4 - Comparaison entre premier ancêtre concept WordNet et tous les ancêtres concepts MeSH

Comme nous l'avons dit dans les chapitres précédents, nous avons trouvé 167 concepts via la comparaison de la première ancêtre concept en commun, mais cela ne permet pas vraiment de trouver le bon concept père pour des paires de concepts intégrables. Étant donné que le temps manque, on laisse le typologie n'a pas encore résoudre mais avec la pensée de solution. Nous pensons que la comparaison de paire concept n'est pas simplement entre premier ancêtre concept WordNet et tous les ancêtres concepts MeSH, il fait bien de comparer tous les ancêtres concepts WordNet avec tous les ancêtres concept MeSH. Et nous pensons ce résultat sera infinitive comme cet algorithme.

5.5 Typologie 5 – La troisième forme d'identifiant de concept MeSH

La troisième forme d'identifiant du concept MeSH correspond aux termes composés avec les virgules. Ce genre de terme nécessite un traitement et une définition concrète.

Dans nos approches, nous avons utilisé le mot avant la première virgule et recherché le même concept dans WordNet. Après avoir eu un aperçu manuel, la plupart des termes avant les premières virgules correspondaient au concept hyperonyme de ce concept. Ainsi, nous pourrions utiliser ce genre de terme pour chercher son concept hyperonyme. S'il existe dans WordNet, nous considérons que ce concept peut être aussi intégré.

Nous avons ainsi trouvé 1 698 concepts co-existants. Néanmoins, les mots avant la première virgule ne figurent pas tous dans MeSH, aussi avons-nous également décidé d'utiliser les numéros d'arbre pour effectuer ce traitement. En fin de compte, la combinaison des méthodes a permis de trouver 789 concepts qui co-existent dans WordNet et MeSH. Et dans ces 1 698 concepts, il existe encore 44 concepts spécifiques que nous regrouperons dans la prochaine typologie. Une combinaison de traitement est le premier mot avant la première virgule et le numéro d'arbre ne permet pas de traiter ce genre d'identifiant.

5.6 Typologie 6 - La quatrième forme d'identifiant de concept MeSH

Pour ce genre de quatrième forme d'identifiant de concept MeSH, prenons un exemple :

"3,4-Dichloro-N-methyl-N-(2-(1-pyrrolidinyl)-cyclohexyl)-benzeneacetamide, (trans)-Isomer", traiter ce genre de concept prendra un temps considérable. C'est la raison pour laquelle nous avons utilisé l'intension et extension de concept, dont le deuxième algorithme pour identifier les co-concepts. Ce genre de typologie a besoin d'un traitement spécifique, par exemple, éliminer tous les tirets au milieu, néanmoins, nous n'avons pas d'expert médical. Ce genre de chaîne contient probablement un sens spécifique, ainsi que ce genre de chaîne n'a pas une explication explicite. Mais ce genre de chaîne pourra être intéressante à traiter. Ainsi, nous ne regroupons que ici comme une typologie.

5.7 Typologie 7 - Out of Vocabulary(OOV) durant l'entraînement de modèle dans le deuxième algorithme

Dans cette typologie, cela fait longtemps que ce genre de problème dans la section de Word2Vec. Il existe également de nombreuses solutions proposées par les communautés dans NLP, comme donner un vecteur random, apporter autant de vocabulaire que possible, ou bien donner un vecteur déjà entraîné par le modèle. Ainsi, nous avons choisi la dernière solution. Nous avons également essayé la deuxième solution proposée par la communauté du secteur, mais cela ne fonctionne pas.

Chapitre 6

Conclusion

Ce stage a été très enrichissant pour moi car il m'a permis de découvrir dans le détail le secteur de la NLU, en plus d'enrichir mes connaissances en analogie du concept entre les ontologies et en Machine Learning. J'ai également compris dans quelle mesure il est possible d'intégrer deux ontologies et ai pu me conforter au sujet de mes compétences en NLP(traitement automatique du langage naturel) et en informatique. Ses acteurs, contraintes de Matching deux ontologie pour le grand public et il m'a permis de participer concrètement à ses enjeux au travers de mes missions variées comme celle du réalisation de browser et API afin une maintenance de dictionnaire, celle du ajouter les "common knowledge" en médecine que j'ai particulièrement apprécié. à travers ces deux missions à la professionnel et à la recherche, deux missions m'orienter vers les métiers d'ingénieur qui me convienne mieux.

Après avoir fini les recherches sur les concepts intégrable, via trois algorithmes : basé sur les morphologies, basée sur la sémantique, basé sur la structure d'ontologie. on pourra utilise tous ces features sur une réflexion d'intégration de concept de partie générale. par exemple : on a utilisé les définitions de concepts à faire une Matching, après, on pourra utiliser aussi les définitions de concepts à trouver la partie de générale dans un concept. Car ces trois algorithmes filtre ou définir les concepts fusionable. cela veut dire tous ces concepts intégrable contient tous les features nécessaires à être intégrable. par exemple : les features sémantique : l'extension et l'intension de concept, les features lexicaux : l'identifiant de terme, les features structurelle : la comparaison de premier ancêtre concept commun.

Même si on a trois partie n'a pas réalisé, tel que quatrième algorithme, intégration d'ontologie et évaluation de la qualité d'intégration. Mais on a noter notre pensée sur ce rapport afin une future travail. Dans la future, on pourra non seulement intégrer les ontologies dans la domaine de biomédical, et aussi autre domaine, par exemple informatique, finance, etc. Et aussi autre difficulté comme les contrains ce qu'on a utilisé qui n'existe pas dans autre ontologie mal structuré. Fort de cette expérience et en réponse à ses enjeux, j'aimerai beaucoup par la suite essayer de m'orienter via un prochain travail, vers le secteur ingénieur d'ontologie avec des acteurs de petites tailles, et un important développement d'avenir.

Annexes

Annexe 1

1 JPA utilisé dans browser

@Entity : Pour définir tous les tableaux dans base de données.

@ID : Pour établir tous identifiant dans base de données.

@Column(name = attribut name dans le tableau)

par exemple : Rel :name, source_id, target_id, domain_id.

@GeneratedValue(strategy = GenerationType.IDENTITY)

@EmbeddedId

@Embeddable

@OneToMany(mappedBy = "word", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)

@OneToMany(mappedBy = "word", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)

@OneToMany(mappedBy = "word", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)

@OneToOne(cascade = CascadeType.ALL)

@Repository

@Controlle

2 Status utilisé dans API

1. La règle : 204 (« Pas de contenu ») doit être utilisée lorsque le corps de la réponse est intentionnellement vide. Le code d'état 204 est généralement envoyé en réponse à

une demande PUT, POST ou DELETE, lorsque l'API REST refuse de renvoyer tout message d'état ou toute représentation figurant dans le corps du message de réponse. Une API peut également envoyer 204 en conjonction avec une demande GET pour indiquer que la ressource demandée existe, mais ne dispose d'aucune représentation d'état à inclure dans le corps.

2. La règle : 202 (« accepté ») doit être utilisée pour indiquer le démarrage réussi d'une action asynchrone. Une réponse 202 indique que la demande du client sera traitée de manière asynchrone. Ce code d'état de réponse indique au client que la demande semble valide, mais il peut toujours avoir des problèmes une fois qu'elle a finalement été traitée. Une réponse 202 est généralement utilisée pour les actions dont le traitement prend beaucoup de temps. Les ressources du contrôleur peuvent envoyer une réponse 202, mais les autres types de ressources ne le devraient pas.

Annexe 2

1 Terminologie d'ontologie

Ces notions sont venues de livre « *Ontology Matching* »[18]

« *Matching* » correspond au processus de recherche de relations ou de correspondances entre des entités d'ontologies différentes.

« *Alignement* » est un ensemble de correspondances entre deux ou plusieurs ontologies (par analogie avec l'alignement de séquence moléculaire). L'alignement est la sortie du processus de correspondance.

« *Correspondance* » est la relation qui existe, ou est supposée être établie selon un alignement particulier, entre des entités d'ontologies différentes. Ces entités peuvent être aussi différentes que des classes, des individus, des propriétés ou des formules. Certains auteurs utilisent le terme « *mappage* » à la place. Cependant, il ne sera pas utilisé dans ce sens dans cet ouvrage.

« *Anchor* » est une correspondance entre deux entités ontologiques, qui sont généralement requises avant d'effectuer d'autres tâches, telles que le raisonnement, le partitionnement et la mise en correspondance.

« *Mapping* » est la version orientée ou dirigée d'un alignement : elle mappe les entités d'une ontologie à au plus une entité d'une autre ontologie. Ceci est plus conforme à la définition mathématique d'un mapping qu'à celle d'une relation générale. La définition mathématique exigerait en principe que l'objet mappé soit égal à son image, c'est-à-dire que la relation est une relation d'équivalence. Un mappage peut être perçu comme un ensemble de règles de mappage toutes orientées dans la même direction, c'est-à-dire d'une ontologie à l'autre, et telles que les éléments de l'ontologie source apparaissent une seule fois.

« *Mapping rule* » est une correspondance qui mappe une entité d'une ontologie en une autre à partir d'une autre ontologie.

« *Ontology merging* » est la création d'une nouvelle ontologie à partir de deux ontologies sources, pouvant se chevaucher. Les ontologies initiales restent inchangées. L'ontologie fusionnée est supposée contenir la connaissance des ontologies initiales, comme les conséquences de chaque ontologie sont les conséquences de la fusion. Ce concept est

étroitement lié à celui de l'intégration de schéma dans les bases de données.

« *Ontology integration* » se réfère à l'inclusion dans une ontologie d'une autre ontologie et d'assertions exprimant le lien entre ces ontologies, généralement sous la forme d'axiomes de pont. L'ontologie intégrée est supposée contenir la connaissance des deux ontologies initiales. Contrairement à la fusion, la première ontologie est inchangée, tandis que la seconde est modifiée.

« *Bridge axioms* » ou « *articulation axioms* » sont des formules, dans un langage d'ontologie, qui exprime les alignements de manière à ce qu'il soit possible d'intégrer les entités d'une ontologie l'une à l'autre. Les axiomes de pont sont à la base de la fusion d'ontologies lorsque celles-ci sont exprimées dans le même langage.

« *Ontology translation* » est le processus de transformation d'une ontologie d'un langage ontologique à un autre. Par extension, il s'agit d'un programme de traduction d'ontologies. La transformation d'ontologie est le processus d'expression des entités d'une ontologie par rapport aux entités d'une autre ontologie, c'est-à-dire que les relations entre les entités de la première ontologie et celles de la seconde sont ajoutées à la première ontologie. Les conséquences initiales de la première ontologie restent donc les conséquences du résultat de la transformation. Les deux ontologies initiales sont inchangées et une troisième ontologie, le résultat de la transformation, est créée. Par extension, c'est un programme qui transforme les ontologies.

« *Data translation* » est le processus de transformation de données ou d'instances d'une seule des données correspondantes ou des instances exprimées par rapport à une autre ontologie. Par extension, c'est un programme qui traduit les données.

« *Data interlinking* » est le processus d'établissement de liens explicites, par exemple d'équivalences, entre instances de différentes sources de données.

« *Mediation* » consiste à interfacier deux composants logiciels en alternant dynamiquement les flux d'information entre ceux-ci. Par extension, un médiateur est un programme effectuant une médiation. Dans la composition du service Web, un médiateur traduit la sortie d'un service en entrée d'un autre : il effectue ainsi la traduction des données. Dans les applications de répondeur de requêtes, il s'agit d'une double paire de traductions qui transforment la requête d'une ontologie à une autre et qui traduit les réponses.

« *Networks of ontologies* » sont constitués d'un ensemble d'ontologies et d'un ensemble d'alignements entre ces ontologies. Les ontologies peuvent être liées pour plusieurs raisons : elles peuvent être complémentaires ; il peut s'agir de deux ontologies indépendantes du point de vue du domaine, comme les ventes et pneus, le raffinement ; une ontologie de domaine peut être spécialisée dans une ontologie de niveau supérieur ; ou elles peuvent être complémentaires, par exemple une version remplaçant une autre version ou deux ontologies sur le même domaine.

« *Linked data* », également appelées Web de données, constitue un espace de don-

nées global composé de sources de données structurées publiées sur le Web, par exemple au format RDF, qui sont explicitement liées.

« Ontology version » est l'ontologie résultant de l'application de modifications à une ontologie.

« Ontology reconciliation » est un processus qui harmonise le contenu de deux ou plusieurs ontologies, nécessitant généralement des modifications d'un des deux côtés, voire des deux côtés (Hameed et al. 2004). Dans ce cas, on ne parle pas de fusion des ontologies, mais de co-évolution. La réconciliation des ontologies peut être effectuée dans le but de fusionner deux ontologies ou de les rendre indépendantes.

2 Technique élémentaire Matching Ontologie

2.1 Techniques basées sur les chaînes

Les techniques basées sur les chaînes sont souvent utilisées pour faire correspondre les noms et les descriptions de noms d'entités d'ontologie. Ces techniques considèrent les chaînes comme des séquences de lettres dans un alphabet. Elles sont généralement basées sur la supposition suivante : plus les chaînes sont similaires, plus elles sont susceptibles de dénoter les mêmes concepts. Habituellement, les fonctions de distance associent une paire de chaînes à un nombre réel, de sorte qu'une valeur plus petite indique une plus grande similarité entre les chaînes. Parmi les exemples de techniques basées sur les chaînes largement utilisées dans les systèmes de mise en correspondance, citons les préfixes, les suffixes, les distances d'édition et la similarité n-gramme.

2.2 Techniques basées sur le langage

Les techniques basées sur la langue considèrent les noms comme des mots dans une langue naturelle, par exemple l'anglais. Elles sont basées sur des techniques de traitement du langage naturel exploitant les propriétés morphologiques des mots d'entrée.

Elles s'appliquent généralement aux noms d'entités avant l'exécution de techniques basées sur des chaînes ou du lexique afin d'améliorer leurs résultats. Cependant, nous considérons ces techniques basées sur le langage comme une classe distincte de techniques d'alignement, car elles peuvent être étendues naturellement, par exemple dans un calcul de distance (en comparant les chaînes résultantes ou les ensembles de chaînes).

Cette classe englobe maintenant la classe de ressources linguistiques de la première édition, qui couvrait l'utilisation de ressources linguistiques, telles que des lexiques ou des thésaurus spécifiques à un domaine, pour faire correspondre les mots (dans ce cas, les noms d'entités d'ontologie sont considérés comme des mots d'une langue naturelle). sur la base de relations linguistiques entre eux, par exemple des synonymes, des hyponymes.

2.3 Techniques basées sur les contraintes

Les techniques basées sur les contraintes sont des algorithmes qui traitent des contraintes internes appliquées aux définitions d'entités, telles que les types, la cardinalité (ou la multiplicité) des attributs et les clés.

2.4 Techniques informelles basées sur les ressources

Les ontologies peuvent être liées à des ressources informelles, telles que l'annotation de pages d'encyclopédie ou d'images. Les techniques informelles basées sur les ressources couvrent les techniques utilisées pour déduire les relations entre entités ontologiques en fonction de la relation qui existe entre elles et ces ressources.

En règle générale, deux classes annotant le même ensemble d'images peuvent être considérées comme équivalentes. Ces techniques exploitent souvent l'analyse de données et les approches statistiques, ainsi que les approches qui exploitent un corpus d'entités apparentées pour trouver des régularités et des différences entre elles. Cette classe de techniques couvre une partie de la classe Analyse de données et techniques statistiques de la première édition traitant des extensions (correspondance basée sur une instance).

2.5 Techniques basées sur les ressources formelles

Les techniques formelles basées sur les ressources tirent parti des ontologies externes pour effectuer la mise en correspondance. Généralement, ils composent des alignements entre les ontologies à appariement provenant d'une ou plusieurs ontologies externes. Plusieurs adaptateurs d'ontologies basés sur le contexte utilisant des ressources formelles ont été proposés ces dernières années, en utilisant des ontologies spécifiques à un domaine, des ontologies de niveau supérieur, des données liées et d'autres ressources.

Cette classe de techniques couvre et généralise les ontologies formelles et les classes de réutilisation des alignements de niveaux supérieurs et spécifiques à un domaine de la première édition de ce livre. Les deux classes se sont concentrées sur différents aspects des techniques formelles basées sur les ressources, à savoir le type d'ontologies ou le type d'alignements utilisés en tant que ressources externes.

3 Techniques au niveau de la structure

Contrairement aux techniques au niveau élément, les techniques au niveau structure considèrent les entités ontologiques ou leurs instances pour comparer leurs relations avec d'autres entités ou leurs instances.

3.1 Techniques basées sur les graphes

Les techniques basées sur les graphes sont des algorithmes de graphes qui considèrent les ontologies en entrée (y compris les schémas de base de données et les taxonomies) comme des graphes étiquetés.

Habituellement, la comparaison des similitudes entre une paire de nœuds des deux ontologies repose sur l'analyse de leurs positions dans les graphiques. L'intuition derrière cela est que, si deux nœuds de deux ontologies sont similaires, leurs voisins doivent également être similaires. Outre les techniques purement basées sur les graphes, il existe d'autres techniques plus spécifiques basées sur la structure, telle que l'utilisation d'arbres.

3.2 Techniques basées sur la taxonomie

Les techniques basées sur la taxonomie sont également des algorithmes de graphes qui ne prennent en compte que la relation de spécialisation. L'intuition qui sous-tend les techniques taxonomiques est que la spécialisation relie des termes qui sont déjà similaires (interprétés comme un sous-ensemble ou un surensemble les uns des autres), de sorte que leurs voisins peuvent également être similaires.

3.3 Techniques basées sur des modèles

Des algorithmes basés sur un modèle (ou fondés sémantiquement) gèrent l'entrée en fonction de son interprétation sémantique, par exemple, la sémantique de la théorie des modèles. L'intuition est que si deux entités sont identiques, elles partagent les mêmes interprétations. Ainsi, ce sont des méthodes déductives bien fondées. Des exemples sont les techniques de raisonnement de la satisfiabilité propositionnelle et de la logique de description.

3.4 Techniques basées sur des instances

Les techniques basées sur les instances sont celles qui comparent des ensembles d'instances de classes afin de décider si ces classes correspondent ou non. Ils peuvent être basés sur un simple raisonnement basé sur la théorie des ensembles ou sur des techniques d'analyse et de statistique plus élaborées. Ils aident à regrouper des éléments ou à calculer les distances entre eux.

À partir de techniques d'analyse de données, nous discutons de la classification à distance, de l'analyse de concept formel et de l'analyse de correspondance ; à partir de méthodes d'analyse statistique, nous considérons des distributions de fréquence. Nous excluons de cette catégorie les techniques d'apprentissage, qui nécessitent un échantillon du résultat, c'est-à-dire l'alignement.

4 L'état de l'art d'une même ontologie

Nous supposons que les données dans les sources d'informations sont décrites par les propriétés et sont organisées dans une hiérarchie taxonomique (sous-classe-superclasse) basée sur l'ontologie de la source. Ces méthodes sont principalement utilisées lors de la phase de calcul de la similarité entre des concepts au sein d'une même ontologie.

Nous présentons un état de l'art pour un ensemble de méthodes basé sur la même ontologie issu d'article "*Identification of New Parameters for Ontology Based Semantic Similarity Measures*"[9] et "*Ontology Mapping : The State of the Art*"[10]. Car notre cas s'applique uniquement aux noms (et aux verbes dans WordNet) et aux relations Is-A. Des méthodes basé sur la même ontologie servent aussi la partie d'évaluation d'intégration des concepts à l'avenir. Ainsi, on présente également ces méthodes.

Les options précédentes soit C l'ensemble des concepts d'une taxonomie IS-A. On voulait mesurer la similarité de deux concepts $\{c1, c2\} \in C$.

4.1 Approche basée sur Structure des Graphes

Cet approche permet de mesurer la similarité entre deux termes (concepts) en fonction de la longueur du chemin reliant les termes et de la position des termes dans la taxonomie. C'était une technique basique pour mesurer la similarité, elle calcule la distance minimale entre les connexions de bout en bout dans leur ontologique associé par un lien «is-a» (ici, «hyperonyme» et «hyponyme» dans WordNet et MeSH). À mesure que la longueur du trajet augmente, la similarité entre les termes tend à diminuer. Si deux termes sont fermés, leur distance doit être faible et la similarité élevée.

Dans les années 1980, Rada et al. (1989)[15] ont exprimé la distance taxonomique de deux concepts définis dans un arbre taxonomique en fonction du chemin le plus court qui les relie.

L'auteur désigne $sp(u, isa^*, v)$ comme étant le chemin le plus court entre deux concepts u et v , c'est-à-dire le chemin de longueur minimale dans u, isa^*, v . La longueur d'un chemin a donc été définie comme la somme des poids associés aux arêtes qui composent le chemin. Lorsque les arêtes ne sont pas pondérées, nous nous référons à la stratégie de comptage des arêtes : la longueur du chemin le plus court correspond au nombre d'arêtes qu'il contient. La distance taxonomique est donc définie par :

$$dist_{Rada}(u, v) = sp(u, isa^*, v) \quad (6.1)$$

Plusieurs problèmes liés aux techniques du chemin le plus court ont été formulés. La stratégie de comptage des arêtes, ou plus généralement toute approche du chemin le plus court avec une épaisseur de carre uniforme, a été critiquée du fait que la distance représentée par une arête reliant deux concepts ne tient pas compte des spécificités de la pertinence des concepts. Par exemple dans WordNet, on notera que la distance entre la plante et l'animal est de 2 (leur parent commun est une chose vivante), et la distance entre le zèbre et le cheval est également de 2 (leur parent commun est l'équin). Pourtant, intuitivement, le cheval et le zèbre semblent plus étroitement liés que les plantes et les animaux.

4.1.1 Les Méthodes Variantes

1. Li et al.(2003) [12]

La mesure de similarité suivante, qui a été dérivée intuitivement et empiriquement, combine la longueur de chemin la plus courte entre deux concepts $c1$ et $c2$, L , et la profondeur dans la taxonomie du concept commun le plus spécifique, c , H , dans une fonction non linéaire.

$$Sim_{Li}(c1, c2) = \frac{-LeH - e - H}{e\Delta eH + e - H} \quad (6.2)$$

Où $\Delta \geq 0$ et $e > 0$ sont des paramètres mettant respectivement à l'échelle la contribution de la longueur et de la profondeur du chemin le plus court. D'après Li et al, les paramètres optimaux sont $\Delta = 0,2$ et $e = 0,6$. Cette mesure est motivée par le fait que les sources d'informations sont infinies dans une certaine mesure, tandis que les humains comparent la similarité des mots avec un intervalle fini allant de complètement similaire à aucunement similaire. Intuitivement, la transformation entre un intervalle infini en un intervalle fini n'est pas linéaire. Il est donc évident que cette mesure se situe entre 1 (pour des concepts similaires) et 0.

2. Wu et Palme (1995) [24]

Cette mesure de similarité considère la position des concepts $c1$ et $c2$ dans la taxonomie par rapport à la position du concept commun le plus spécifique c . Étant donné que chaque concept peut avoir plusieurs parents, deux concepts peuvent partager les parents de plusieurs manières. Le concept commun le plus spécifique correspond au parent commun associé au nombre minimal de liens IS-A avec les concepts $c1$ et $c2$.

$$Sim_{W\&P}(c1, c2) = \frac{2H}{N1 + N2 + 2H} \quad (6.3)$$

Où $N1$ et $N2$ sont le nombre de liens IS-A de $c1$ et $c2$ respectivement vers le concept commun le plus spécifique c , et H est le nombre de liens IS-A de c à la racine de la taxonomie. Les scores sont compris entre 1 (pour des concepts similaires) et 0.

4.1.2 Shortest Path

Cette mesure est une variante de la méthode de la distance et est principalement conçue pour fonctionner avec des hiérarchies. C'est une mesure simple et puissante dans les réseaux sémantiques¹ hiérarchiques. Soit $c1$ et $c2$ deux concepts pour lesquels, la mesure de similarité dans la structure hiérarchique peut être formulée comme suit :

$$sim(c_1, c_2) = 2 * Max(c_1, c_2) - SP \quad (6.4)$$

1. Dans un réseau sémantique, processus par lequel est diffusée l'activation d'un concept aux concepts associés. Modèles d'organisation des concepts en mémoire sémantique. Les concepts sont représentés par des nœuds dans le réseau et les relations entre concepts sont représentées par des liens (ou arcs) entre les nœuds.

Où Max est la longueur maximale du chemin entre $c1$ et $c2$ dans la taxonomie et SP est le chemin court reliant (nombre minimal de liens) les concepts $c1$ au concept $c2$.

Il existe de nombreuses questions spécifiques sur le réalisme cognitif de la mesure du chemin le plus court, mais il s'agit d'une mesure simple et puissante dans les réseaux sémantiques hiérarchiques.

4.1.3 Weighted Links

Dans le prolongement de la mesure ci-dessus, il est proposé d'utiliser des liens pondérés pour calculer la similarité entre deux concepts.

Le poids d'un lien peut être affecté par : (a) la densité de la taxonomie à ce point, (b) la profondeur dans la hiérarchie, et (c) la force de la connotation entre les nœuds parent et enfant. Ensuite, le calcul de la distance entre deux concepts se traduit par la somme des poids des liens traversés au lieu de les compter.

4.1.4 Hirst and St-Onge Measure

Cette mesure veut que deux concepts, $c1$ et $c2$, sont sémantiquement proches s'ils sont reliés par un chemin qui n'est pas trop long et qui ne change pas trop souvent de direction.

$$sim_{HSO}(c1, c2) = C - SP - k * d \quad (6.5)$$

Où d est le nombre de changements de direction dans la trajectoire et C , k sont constants. Bien que cette mesure donne une perspective différente de la similitude entre deux concepts, elle semble donner de piètres résultats principalement parce qu'elle réside dans sa tendance à errer que dans l'utilisation des relations de concept.

4.1.5 Leacock et Chodorow

La mesure de parenté proposée par Leacock et Chodorow est

$$sim_{ch}(c1, c2) = \log(longueur / (2\Delta D)) \quad (6.6)$$

où la longueur est la longueur du chemin le plus court entre les deux synsets (en utilisant le comptage de nœuds) et D est la profondeur maximale de la taxonomie ? Le fait que cette mesure prenne en compte la profondeur de la taxonomie dans laquelle se trouvent les synsets signifie que le comportement de la mesure est profondément affecté par la présence ou non d'un nœud racine unique.

S'il existe un nœud racine unique, il n'existe qu'une taxonomie : une pour les 15 sous-arbres de MeSH. Cependant, si le nœud racine n'est pas utilisé, il existe 15 sous-arbres différents, chacun avec une valeur différente pour D .

Dans ce cas, il est possible que les synsets appartiennent à plus d'une taxonomie. Par exemple, le synset contenant le terme « douleur » appartient à trois taxonomies : une forme sur les « maladies », une en psychiatrie et psychologie et une en « sciences biologiques ».

Dans un tel cas, la relation est calculée en recherchant le service LCS qui se traduit par le chemin le plus court entre les synsets. La valeur de D est donc la profondeur maximale de la taxonomie dans laquelle se trouve le LCS.

Si le LCS appartient à plus d'une taxonomie, la taxonomie avec la profondeur maximale la plus grande est sélectionnée (c'est-à-dire, la valeur la plus grande pour D).

Les mesures mentionnées ci-dessus sont basées uniquement sur des liens taxonomiques (IS-A) entre concepts, en supposant que les liens dans la taxonomie représentent des distances. Cependant, la densité de termes dans la taxonomie n'est généralement pas constante. En règle générale, des termes plus généraux existent plus haut dans la hiérarchie et représentent un ensemble de nœuds plus petit que le plus grand nombre de termes plus spécifiques qui peuplent un espace beaucoup plus dense situé plus bas dans la hiérarchie.

De plus, dans les taxonomies, il existe une grande variabilité dans ce qui est couvert par un seul lien taxonomique. Par exemple, la valve de sécurité IS-A semble beaucoup plus étroite que la machine à tricoter IS-A. La mesure des liens pondérés peut prendre en compte la force des liens si leur pondération est calculée en conséquence.

Dans ce qui suit, nous présentons des mesures impliquant un contenu d'information, qui semblent mieux fonctionner que des mesures de comptage des contours.

4.2 Approche basé sur les caractéristiques du concept

Cet approche permettent de mesurer la similarité entre deux termes en fonction de leurs propriétés (par exemple, leurs définitions ou « glosses » dans WordNet ou « notes de champ » dans MeSH) ou en fonction de leurs relations avec d'autres termes similaires dans la taxonomie. Les caractéristiques communes ont tendance à augmenter la similarité de deux contextes tandis que les caractéristiques non communes ont tendance à la diminuer.

L'idée principale est de représenter les concepts sous forme de collections de caractéristiques, c'est-à-dire de caractéristiques décrivant les concepts, afin d'exprimer davantage les mesures en fonction de l'analyse de leurs caractéristiques communes et distinctes. Le score des mesures ne sera influencé que par la stratégie adoptée pour déterminer les caractéristiques des concepts et par la stratégie adoptée pour les comparer.

4.2.1 Tversky (1977) [22]

Cette mesure est basée sur les ensembles de description des termes. Nous supposons que chaque terme est décrit par un ensemble de mots indiquant ses propriétés ou caractéristiques. Ensuite, plus les deux termes ont des caractéristiques communes et moins ils ont de caractéristiques non communes, plus les termes sont similaires.

$$Sim_{Tversky}(c1, c2) = \frac{|C1 \cap C2|}{|C1 \cap C2| + |C12| + (K - 1)|C21|} \quad (6.7)$$

Où $C1$, $C2$ correspondent aux ensembles de description des termes $c1$ et $c2$ respectivement et K $[0,1]$ définit l'importance relative des caractéristiques non communes. Cette mesure obtient un score compris entre 1 (pour des concepts similaires) et 0, qui augmente avec les points communs et diminue avec la différence entre les deux concepts. À l'inverse de toutes les mesures présentées ci-dessus, cela n'a rien à voir avec la taxonomie et les sous-utilisateurs des termes, et semble mieux exploiter les propriétés de l'ontologie utilisée.

Dans la mesure présentée ci-dessus, la détermination de K est fondée sur le constat que la similarité n'est pas nécessairement une relation symétrique : À l'inverse des caractéristiques isolées, les caractéristiques communes entre une sous-classe et sa classe supérieure ont une plus grande contribution à l'évaluation de la similarité que les caractéristiques communes dans le sens inverse. Compte tenu de cette hypothèse, il fournit une approche systématique pour déterminer l'asymétrie d'une évaluation de similarité.

4.3 Approche basé sur la théorie de l'information

L'approche théorique de l'information s'appuie sur la théorie de l'information de Shannon (Shannon, 1948). Comme pour la stratégie basée sur les caractéristiques, ces mesures reposent sur la comparaison de deux concepts en fonction de leurs points communs et de leurs différences, définis ici en termes d'informations. Cette approche introduit formellement la notion de saillance des concepts par la définition de leur informativité.

Resnik (1995) définit la similarité de quelques concepts en fonction du CI de leur ancêtre commun, qui maximise une fonction de CI (à l'origine eIC), c'est-à-dire leur ancêtre commun le plus informatif (MICA) :

$$Sim_{Resnik}(u, v) = IC(MICA(u, v)) \quad (6.8)$$

4.3.1 Les méthodes variantes

1. Jiang et al. (1998)

Contrairement aux mesures de similarité ci-dessus, cette mesure est une distance sémantique.

$$Dist_{Jiang}(c1, c2) = -2lnpmis(c1, c2) - (lnp(c1) + lnp(c2)) \quad (6.9)$$

Ainsi, la similitude entre deux concepts $c1$ et $c2$, $sim_Jiang(c1, c2)$, est calculée comme suit : $1 - Dist_Jiang(c1, c2)$. Cette mesure peut donner des valeurs arbitrairement grandes, comme la mesure Resnik, bien qu'en pratique elle ait une valeur maximale de $2 \ln(N)$, où N est la taille du corpus. De plus, il combine le contenu des informations provenant du parent partagé et des concepts comparés, en tant que mesure Lin. Ainsi, cette mesure semble combiner les propriétés des mesures présentées ci-dessus, c'est-à-dire qu'elle nous fournit à la fois des informations sur la taille de l'ontologie et le classement de différentes paires de termes.

4.3.2 Lin

Cette mesure utilise à la fois la quantité d'informations nécessaire pour énoncer le caractère commun de deux termes et l'information nécessaire pour les décrire complètement.

Comme $pmis \geq p(c1)$ et $pmis \geq p(c2)$, les valeurs de cette mesure varient entre 1 (pour des concepts similaires) et 0. Dans ce cas, un terme comparé à lui-même obtiendra toujours 1, masquant ainsi les informations révélées par la mesure de Resnik.

Cependant, la mesure Resnik dépend uniquement du contenu informationnel des parents partagés, et il y a seulement autant de scores distincts que de termes d'ontologie. En utilisant le contenu informationnel des termes comparés et du parent partagé, le nombre de scores discrets est quadratique dans le nombre de termes apparaissant dans l'ontologie, augmentant ainsi la probabilité d'avoir différents scores pour différentes paires de termes. Par conséquent, l'utilisation de cette mesure pour comparer les termes d'une ontologie peut avoir un meilleur classement de similarité que la mesure de Resnik.

4.4 Approches hybrides

Les approches hybrides consistent en une méthode où ces deux approches (voire plus) peuvent être combinées. Des recherches récentes suscitent un intérêt pour ces approches. Dans la période récente, l'auteur Cai et al. a utilisé une technique hybride basée sur Edge et sur l'approche du contenu informationnel.

4.4.1 Knappe

Cette mesure est principalement basée sur le fait qu'il peut y avoir plusieurs chemins reliant deux concepts. Prendre tous les chemins possibles implique une augmentation substantielle de la complexité. Ainsi, l'idée générale met l'accent sur les concepts « partagés » et une mesure de similarité représentant la partie de l'ontologie couvrant les concepts comparés est définie. De plus, il existe une notion de concepts complexes qui permet à un concept d'être constitué de plus d'un terme. Initialement, la décomposition du terme © d'un concept c en un ensemble C est définie, et le développement vers le haut (C) de C est effectué. Le terme décomposition de c est défini comme l'ensemble

de tous les concepts inclus dans c (si c est un concept complexe, sinon cet ensemble ne comprend que c) et tous les attributs de ces concepts. Si, par exemple, le concept initial c est « le chien », la décomposition du terme pourrait être l'ensemble $C = \text{chien, couleur}$. L'expansion vers le haut, (C) , implique les liaisons IS-A de tous les éléments de C .

Soit $u(c)$ l'ensemble des nœuds accessibles depuis c , c'est-à-dire $u(c) = (c)$. Les nœuds accessibles partagés par $c1$ et $c2$ sont $u(c1) \cap u(c2)$. Ensuite, nous considérons les directions ascendante et descendante du graphique comme une généralisation et une spécialisation, respectivement. Trois propriétés principales souhaitables sont prises en compte pour définir la fonction de similarité : (a) le coût de la généralisation doit être nettement supérieur au coût de la spécialisation, ce qui indique que la fonction de similarité ne peut être symétrique (b) le coût des bords transversaux doit être inférieur lorsque les nœuds sont plus spécifiques et (c) une spécialisation plus poussée implique une similarité réduite.

$$\text{sim}_{\text{Knappé}}(c1, c2) = \frac{|u(c1) \cap u(c2)| + (1-\alpha)|u(c1) \setminus u(c2)|}{|u(c1)| + |u(c2)|} \quad (6.10)$$

Où $\alpha \in [0, 1]$ détermine le degré d'influence des généralisations.

Cette mesure obtient un score compris entre 1 (pour les concepts correspondants) et 0. Le but de cette mesure de similarité est d'introduire une évaluation douce plutôt que nette, car nous souhaitons généralement rechercher des valeurs similaires plutôt que identiques à la perfection. De plus, l'idée d'élargir le concept conduit l'alignement de similarité vers une comparaison d'ensembles, en incorporant dans la mesure de similarité, la connaissance représentée par l'ontologie.

Nous examinons si les caractéristiques communes des concepts comparés ont une incidence sur les mesures de similarité et si les différences entre les concepts entraînent une diminution des mesures. De plus, nous pensons que la relation entre les mesures de similarité avec la taxonomie et les relations taxonomiques c'est-à-dire si la position des concepts dans la taxonomie et le nombre de liens IS-A sont pris en compte. Il est également indiqué si les mesures de similarité prennent en compte le contenu informationnel des concepts, qu'ils soient liés ou renvoient des valeurs infinies, qu'ils soient symétriques (ie, $\text{sim}(c1, c2) = \text{sim}(c2, c1)$), et s'ils donnent des perspectives différentes.

4.4.2 Rodriguez et al.

Cette mesure de similarité peut être utilisée à la fois pour des similitudes simples ou croisées. La fonction de similarité détermine des classes d'entités similaires en utilisant des méthodes de mise en correspondance sur des ensembles de synonymes, des voisinages sémantiques et des caractéristiques distinctives qui sont ensuite classées en partie, fonctions et attributs (par exemple, en considérant le terme collège, une fonction consiste à éduquer ; toit et sol, et autres attributs peuvent être des propriétés architecturales). La fonction de similarité est la somme pondérée des valeurs de similarité pour les ensembles de synonymes, les quartiers et les entités.

$$S(ap, bq) = w\Delta Sw(ap, bq) + u\Delta Su(ap, bq) + n\Delta Sn(ap, bq) \quad (6.11)$$

Où w, u et $n \geq 0$ et $w + u + n = 1$. Pour chaque type de caractéristiques distinctives, Sw, Su et Sn , une fonction de similarité $\text{simTversky}(c1, c2)$ est utilisée sur la base du modèle de correspondance des caractéristiques de Tversky.

Les fonctions Sw, Su et Sn représentent la similarité entre les ensembles de synonymes, les fonctions et les voisinages sémantiques entre les classes d'entités a de l'ontologie p et b de l'ontologie q et sont calculées à l'aide de cette équation. Les poids w, u et n sont les poids respectifs de la similarité de chaque composant de spécification. la méthode variant

Les glossaires sont fournis avec chaque ensemble de synonymes (synset) dans la plupart des ontologies, nous fournissant une description du terme signification ou une note de portée pour le terme référé. Ce glossaire est présent dans la plupart des ontologies, y compris MeSH et Wordnet. Ce type d'information doit également être pris en compte pour calculer la similarité entre les termes. Nous proposons de remplacer Feature Matching (Su) par une similarité de brillant calculée à l'aide de la formule suivante :

$$S_{gloss}(ap, bq) = \frac{|A \cap B|}{|A \cup B|} \quad (6.12)$$

Où A et B sont des ensembles contenant des termes forment les glossines des concepts ap, bq .

Dans la méthode d'origine, l'appariement de voisinage sémantique est calculé de la manière suivante : la fonction de similarité place tous les termes du rayon r du terme a (termes similaires supérieurs au seuil de similarité r) dans un seul ensemble et les compare à un autre ensemble contenant des termes dans le même critère. Même rayon de terme b . La similitude entre les deux ensembles est à nouveau calculée à l'aide de l'équation 2.14. Nous proposons la formule suivante pour calculer l'appariement sémantique de voisinage.

$$S_{nm}(ap, bq) = \max_i \frac{|Ai \cap Bi|}{|Ai \cup Bi|} \quad (6.13)$$

Où i dénote des relations (c'est-à-dire des hyponymes, des méronymes partiels, des holonymes, etc., dans l'ontologie Wordnet, alors que dans l'ontologie MeSH, il n'y a que des hyponymes et des hypernymes) qu' ap et bq ont tous les deux. Ai, Bi sont des ensembles contenant tous les termes dérivés de la i -ième relation des concepts ap, bq . En d'autres termes, nous proposons de prendre le maximum de similitude entre les deux termes dans une partie du quartier et non dans toute la région. C'est l'idée de base, mais alors que dans MeSH, il n'y a qu'une relation d'hyponymie et d'hyperonyme (plus définition-description) à partir des relations notées Wordnet pour un concept, l'équation ci-dessus dérive d'une seule valeur de similarité, celle de descendant ou d'antécédent termes.

Bibliographie

- [1] Paola Velardi. Aldo Gangemi, Roberto Navigli. The ontowordnet project : Extension and axiomatization of conceptual relations in wordnet. 2003. Published in CoopIS/DOA/ODBASE 2003.
- [2] Martial BANON. *JPA et Java Hibernate - Apprenez le mapping objet-relationnel (ORM) avec Java*. Editions ENI, 2017.
- [3] Tomas Mikolov. Kai Chen. Greg Corrado. Jeffrey Dean. Efficient estimation of word representations in vector space. 2013. arXiv :1301.3781v3.
- [4] Antonio M. Rinaldi. Enrico G. Caldarola. An approach to ontology integration for ontology reuse. 2016. Department of Electrical Engineering and Information Technologies, University of Naples, Federico II, Napoli, Italy Institute of Industrial Technologies and Automation, National Research Council, Bari, Italy IKNOS-LAB Intelligent and Knowledge Systems, University of Naples Federico II, LUPT, Napoli, Italy.
- [5] Euripides G.M. Petrakis et al. X-similarity : Computing semantic similarity between concepts from different ontologies. 2006. Department of Electronic and Computer Engineering Technical University of Crete (TUC).
- [6] Fausto Giunchiglia et al. S-match : an algorithm and an implementation of semantic matching. 2004. In Proceedings of the European Semantic Web Symposium, LNCS 3053, pp. 61-75, 2004.
- [7] Jetendr Shamdasani et al. Semantic matching using the umls. 2009. CCCS Research Centre, CEMS Faculty, University of the West of England Coldharbour Lane, Frenchay, Bristol BS16 1QY, UK.
- [8] P.XIAO et al. Conceptual representations for computational concept creation. 2018. ACM Computing Surveys, Vol. 1, No. 1, Article 1.
- [9] Shivani et al. Identification of new parameters for ontology based semantic similarity measures. 2019. Department of Computer Science Engineering.
- [10] Yannis et al. Ontology mapping : The state of the art. 2005. Department of Electronics and Computer Science.
- [11] Roy Thomas Fielding. Architectural styles and the design of network-based software architectures. 2000. UNIVERSITY OF CALIFORNIA, IRVINE.
- [12] Bandar Z.A Li, Y. and D. McLean. An approach for measuring semantic similarity between words using multiple information sources. 2003. [IEEE Transactions on Knowledge and Data Engineering, 15(4) :871-882, July/August.].
- [13] Auteur Lui. Comparing sentence similarity methods. <<http://nlp.town/blog/sentence-similarity/>>.

- [14] Mark Masse. *REST API Design Rulebook*. O'Reilly, 2011.
- [15] Mili H. Bicknell E. Rada, R. and M. Blettner. Ieee transactions on systems, man, and cybernetics. 1989. [19(1) :17-30, January/February].
- [16] Antonio M. Rinaldi. A multimedia ontology model based on linguistic properties and audio-visual features. 2013. DIETI - Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione 80125 Via Claudio, 21, Napoli, Italy.
- [17] Sagot. The lefff, a freely available and large-coverage morphological and syntactic lexicon for french. 2010. In Proceedings of the 7th international conference on Language Resources and Evaluation (LREC 2010), Istanbul, Turkey.
- [18] Jérôme Euzenat & Pavel Shvaiko. *Ontology Matching (2nd Edition)*. Springer, 2013.
- [19] Waleed Ammar. George Mulcaire. Yulia Tsvetkov. Guillaume Lample. Chris Dyer. Noah A. Smith. Massively multilingual word embeddings. 2016. arXiv :1602.01925v2 [cs.CL].
- [20] Stefan Janaqi Jacky Montmain Sébastien Harispe, Sylvie Ranwez. *Semantic Similarity from Natural Language and Ontology analysis*. 2017.
- [21] Arora S. Liang Y. Ma T. A simple but tough-to-beat baseline for sentence embeddings. 2017. Int. Conf. Learn. Represent. (Toulon, France), 1–16.
- [22] A Tversky. Features of similarity. 1977. Psychological Review, 84(4), 327-352.
- [23] David Guthrie. Ben Allison. Wei Liu. Louise Guthrie. Yorick Wilks. A closer look at skip-gram modelling. 2006. NLP Research Group, Department of Computer Science, University of Sheffield.
- [24] Z. Wu and M. Palmer. Verb semantics and lexical selection. 1994. [In Proceedings of the 32nd Annual Meeting of the Associations for Computational Linguistics (ACL'94), pages 133].